TCMS — Architecture & Complete Working Guide

Every layer, endpoint, page and file — explained. Next.js 16 · React 19 · Prisma 6 · PostgreSQL (JSONB)

Welcome! This folder documents **every layer, every endpoint, every page and every file** of the TCMS (Test Case Management System) project. Start with `01` and read in order for a full understanding, or jump straight to what you need.

#	Document	What it covers
01	Overview & Architecture	What TCMS is, tech stack, system diagram, request lifecycle, boot sequence, directory map, env vars, conventions
02	Database Schema	All 14 tables, every column, relations & cascade rules, JSONB design, status lifecycles, deletion semantics
03	Authentication & Permissions	JWT session mechanics, token refresh dance, roles, full permission matrix, endpoint protection map, security notes
04	API Reference	Every endpoint: params, validation, business rules, side effects (audit + notifications), response shapes, quirks
05	Frontend Architecture	Zustand stores, React Query key inventory, the fetch layer, error/toast system, shared components, theming
06	Pages: Login & Dashboard	AuthScreen (login/register) and the dashboard (KPIs, charts, activity feed)
07	Pages: Modules & Test Cases	Module tree, test-case table + create dialog, case detail with versioning
08	Pages: Review Workflow	The review state machine, queue views, verdict dialogs, module bulk review + auto-completion
09	Pages: Bugs, Tasks, Reports	Kanban drag & drop, daily work log editor, team analytics
10	Pages: Activity, Users, Notifications, Profile	Audit trail UI, notification center, admin console, self-service settings
11	End-to-End Flows	8 complete click-to-database stories + the 7 cross-flow invariants
12	Realtime & Deployment	WebSocket mini-service protocol, Vercel + Neon topology, ops quick reference
13	File Index	Every file in the project explained in one or two lines

Other documentation in this project

- `../DEPLOYMENT-GUIDE.md` — deploy the app online in ~25 minutes without installing anything (Neon + GitHub + Vercel, in English).
- `../README.md` — project overview, quick start, demo accounts.

Reader's tip: if you read only three files, make them `01` (the map), `11` (the flows — how everything connects) and `04` (the precise behavior of the backend).

01 · Overview & Architecture

> **TCMS — Test Case Management System.** This document explains what the project is, how it is structured, and how a request travels from the browser to the database and back. Read this first; the other documents in `docs/` go deeper into each layer.

---

1. What TCMS does

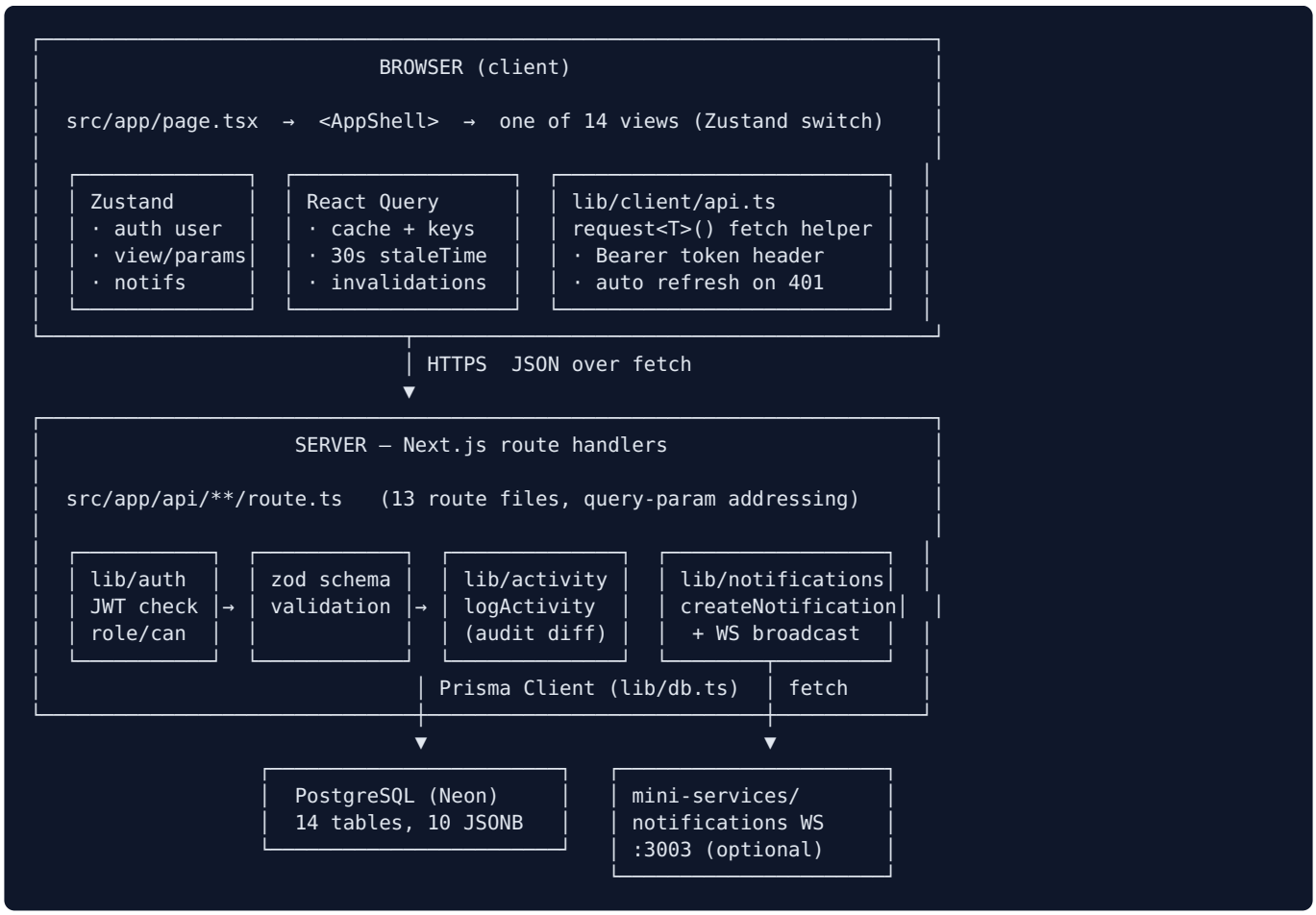
TCMS is a full-stack web application for QA (quality assurance) teams. In one place a team can:

- **Organise** product features into a nested **module tree** (e.g. `Authentication → Login`).
- **Author test cases** with step-by-step instructions, priorities, tags and expected results.
- **Run peer reviews** — a test case or a whole module is submitted to a reviewer who approves, fails or requests revision; every submission is a numbered "cycle" and every edit creates a new version with a stored snapshot.
- **Track bugs** on a drag-and-drop Kanban board, optionally linked to test cases.
- **Log daily work** (task reports: what was done, how many minutes, linked to modules/cases/bugs).
- **See everything that happened** through in-app notifications and a full audit trail (activity log with before/after field diffs).
- **Administer users** with four roles and a permission matrix.

2. Technology stack (with exact versions from `package.json`)

Layer	Technology	Version	Role
UI framework	**Next.js** (App Router)	^16.1.1	Serves the app, hosts the API routes
UI library	**React**	^19.0.0	Component model
Language	**TypeScript**	^5	Types everywhere
Styling	**Tailwind CSS** + shadcn/ui (Radix primitives)	v4	Look & feel
Client state	**Zustand**	^5.0.6	Auth session, current view, notifications
Server state	**TanStack React Query**	^5.82.0	Fetching, caching, invalidation
Charts	**Recharts**	^2.15.4	Dashboard + reports
Drag & drop	**@dnd-kit/core**	^6.3.1	Bug Kanban board
ORM	**Prisma**	^6.11.1	Typed database access
Database	**PostgreSQL** (Neon-compatible)	17+	Storage; 10 native JSONB columns
Auth	**jsonwebtoken** + **bcryptjs**	9.0.3 / 3.0.3	JWT sessions, password hashing
Validation	**zod**	^4.0.2	Request body validation
Toasts	**sonner**	^2.0.6	User feedback
Realtime	**socket.io / socket.io-client**	4.8.x	Optional live notifications (mini-service)

3. High-level architecture



Three things make this architecture distinctive; keep them in mind throughout the docs:

1. **It is a single-page application on one URL.** Next.js App Router has exactly one page (`src/app/page.tsx` → `<AppShell/>`). Which "page" you see is decided by a Zustand store (`useViewStore.current`), not by the URL. All navigation, deep links and role-gated menus are plain React state (documented in `05-frontend-architecture.md`).
2. **The API is addressed by query parameters, not path segments.** One route file per domain (e.g. `api/test-cases/route.ts` handles everything) and the operation is chosen via `?id=`, `?action=` or (for auth) `?path=`. Example: `PUT /api/test-cases?id=abc&action=submit-review`.
3. **Every mutating request writes an audit record.** Routes call `logActivity()` which stores before/after snapshots and a computed field-level `diff` into the `ActivityLog` table; failures to log never break the main operation.

4. Request lifecycle (a complete example)

What happens when a QA engineer clicks **Submit for Review** on a test case:

```
1. View component (test-case-detail.tsx) — "Submit for Review" dialog
   → testCasesApi.submitReview(id, { reviewerId, dueDate })
2. lib/client/api.ts request<T>()
   → PUT fetch to /api/test-cases?id=…&action=submit-review
   → headers: Content-Type: application/json + Authorization: Bearer <jwt>
3. Server route api/test-cases/route.ts, PUT handler
   a. getCurrentUser()            → reads cookie tcms-access, verifies JWT   (401 if bad)
   b. can('submit_review')        → role check: not viewer                   (403 if bad)
   c. case lookup                 → 404 if missing
   d. reviewer resolution         → body reviewerId, else case.assigneeId,
                                    else first active team_lead/admin
```

```
    e. cycle = max(cycle)+1        → create ReviewRequest, set case → in_review
    f. logActivity(REVIEW_SUBMITTED/test_case, after:{reviewerId, cycle})
    g. createNotification(reviewer, 'review_assigned') → insert Notification row
                                                       → fire-and-forget POST :3003/notify
 4. Response { …review row… }  → 200 JSON
 5. Client: toast.success('Submitted for review')
    → queryClient.invalidateQueries(['reviews…'])  → queue/sidebar badges refetch
 6. Reviewer's browser (if WS connected): socket 'notification' event
    → zustand addNotification → bell badge increments instantly.
      (If WS is unavailable — e.g. on Vercel — the row still exists in the DB and
       appears on next fetch/refresh; nothing breaks.)
```

Every flow in the system follows this same 6-step shape: **view → api client → route (guard → validate → business rule → audit → notify) → response → toast + invalidate → (optional realtime push)**.

5. Boot sequence (what happens from page load to usable app)

```
1. Browser GET /                → Next.js serves the static page shell
2. layout.tsx                   → <Providers> mounts:
     QueryInit                  → React Query client (staleTime 30s, retry 1)
     AuthGate                   → useAuthStore.initialize()
3. initialize():
     read localStorage tcms-user + tcms-access
     if present → GET /api/auth (me) to validate the token server-side
        ok    → user stays logged in (session restored)
        fail  → clearSession() (user must log in again)
4. While !booted → centred spinner
5. AppShell: user ? <shell + sidebar> : <AuthScreen login/register>
6. Sidebar mounts → prefetches review queues (badge), 30s polling
7. useNotificationSocket connects to WS (if user) and seeds notifications list
8. Default view = 'dashboard' → GET /api/stats?action=dashboard
```

6. Repository directory map

```
├── prisma/
│   ├── schema.prisma          14 models, PostgreSQL, 10 Json columns
│   └── database-setup.sql     paste-into-Neon table creation script
├── src/
│   ├── app/
│   │   ├── layout.tsx         root layout: fonts, metadata, <Providers>, toasters
│   │   ├── page.tsx           the single page → <AppShell/>
│   │   ├── globals.css        Tailwind v4 theme tokens (oklch)
│   │   └── api/…              13 route handlers (see 04-api-reference.md)
│   ├── components/
│   │   ├── providers.tsx      React Query provider + auth boot gate
│   │   ├── app-shell.tsx      sidebar + topbar + view switch
│   │   ├── auth-screen.tsx    login / register split screen
│   │   ├── layout/            sidebar.tsx, topbar.tsx
│   │   ├── views/             14 view components = the "pages" (docs 06–10)
│   │   ├── shared/            state.tsx (loading/empty/error), pagination.tsx
│   │   └── ui/                ~40 shadcn/ui primitives (button, dialog, …)
│   ├── hooks/                 use-notification-socket, use-toast, use-mobile
│   ├── lib/
│   │   ├── client/api.ts      FRONTEND data layer: fetch + session + api namespaces
│   │   ├── api.ts             BACKEND response helpers (ok/401/403/404/paginate)
│   │   ├── auth.ts            JWT, password hash, roles, permission matrix
│   │   ├── db.ts              Prisma singleton
│   │   ├── activity.ts        audit logging + JSONB helpers
│   │   ├── notifications.ts   in-app notification + WS broadcast
│   │   ├── serializers.ts     JSONB normalisers for every row type
│   │   ├── format.ts          dates, colors, labels
│   │   ├── errors.ts          client error → toast mapping
│   │   └── utils.ts           cn() class merger
│   ├── store/index.ts         Zustand: auth, view, notifications
│   └── types/index.ts         shared TypeScript domain types
```

```
├── mini-services/notifications/  optional socket.io WS relay (:3003)
├── docs/                          THIS documentation
├── DEPLOYMENT-GUIDE.md            step-by-step online deployment (English)
└── package.json / configs
```

7. Environment variables

Variable	Used by	Purpose
`DATABASE_URL`	Prisma	Pooled PostgreSQL connection string (app queries). On Neon it ends in `-pooler` host, port 6543.
`DIRECT_URL`	Prisma	Direct (non-pooled) string for schema tools.
`JWT_SECRET`	lib/auth.ts	HS256 signing secret for access & refresh tokens. Falls back to a dev default — always set it in production.

Nothing else is required. `PORT` is fixed to 3000 in the npm scripts. The WS mini-service is configured by constant (`PORT = 3003` inside `mini-services/notifications/index.ts`).

8. Conventions used across the codebase

- **String enums**: status/priority/role values are plain strings (`'draft'` , `'P2'` , `'team_lead'`) mirrored in `src/types/index.ts` — the single source of truth for the UI.
- **JSONB columns** are read through `parseJson()` (handles both native JSONB objects and legacy JSON strings) and serialised exactly once per row via `lib/serializers.ts` .
- **Auth transport**: the JWT access token travels in the `Authorization: Bearer` header AND is mirrored into cookie `tcms-access` (server routes read the cookie via `next/headers`). The refresh token lives only in localStorage and is sent in the JSON body of `POST /api/auth?path=refresh` .
- **Feedback**: success → `toast.success(...)` ; failure → `handleApiError(err, fallback)` which maps HTTP status to a friendly message. No optimistic updates anywhere — the UI always waits for the server, then invalidates React Query caches.
- **Time**: stored as UTC `DateTime` , rendered with `en-US` formatters from `lib/format.ts` .

9. Where to read next

Want to understand…	Read
The data model, every table/column/relation	`02-database-schema.md`
Login, tokens, roles, who may do what	`03-authentication-and-permissions.md`
Every endpoint, request/response, side effects	`04-api-reference.md`
The client machine: stores, cache keys, fetch layer	`05-frontend-architecture.md`
How each page/screen works	`06` – `10-page-*.md`
Complete click-to-database stories	`11-end-to-end-flows.md`
Realtime service + Vercel/Neon deployment architecture	`12-realtime-and-deployment.md`
Any single file explained in one paragraph	`13-file-index.md`

02 · Database Schema — every table explained

> Source of truth: `prisma/schema.prisma` (PostgreSQL provider). This document mirrors every model, column, relation, default and index, then explains why it is shaped that way. Apply `prisma/database-setup.sql` (or `npm run db:push`) to create it.

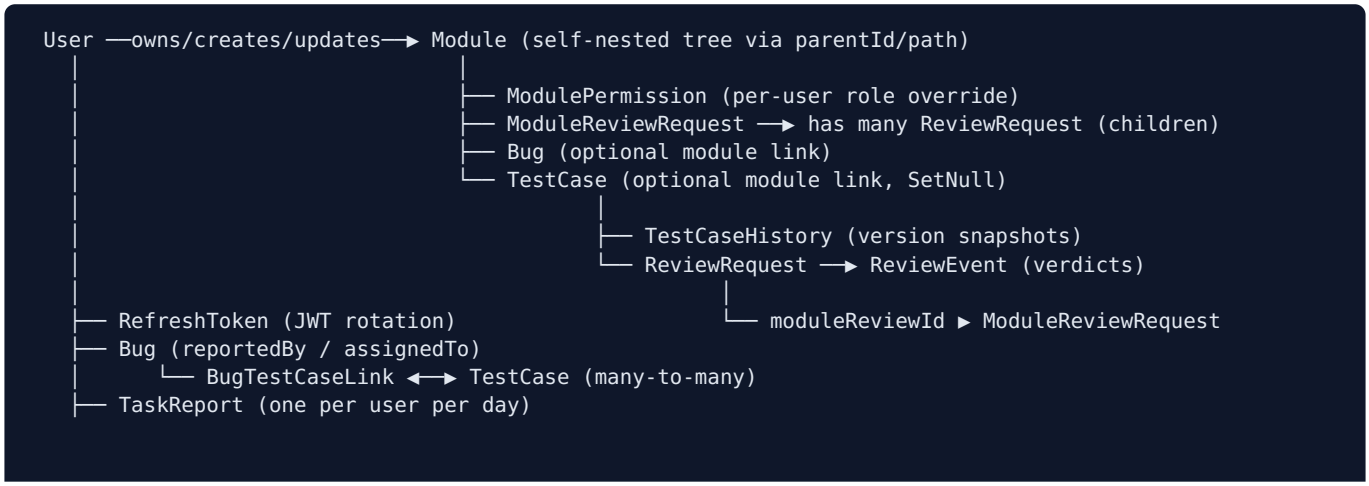
> ⚠️ **SEPTEMBER 2026 RESTRUCTURE:** the database was renamed to strict SQL conventions — snake_case tables/columns and **entity-named primary keys** (`users.user_id`, `modules.module_id`, `test_cases.test_case_id`, … — never a generic `id`), DB-generated uuid PKs (`gen_random_uuid()`). This chapter's prose still explains the SHAPE of every table correctly, but wherever you see `id` or camelCase column names, read the new names from `prisma/schema.prisma` (`@map`/`@@map`) or `prisma/database-setup.sql`. Full mapping: `REFACTORING-REPORT.md` §1.

---

1. Conventions

Convention	Detail
Primary keys	`id String @id @default(cuid())` — collision-safe string ids (except `BugTestCaseLink`, composite PK).
Timestamps	`createdAt DateTime @default(now())` + `updatedAt DateTime @updatedAt` (Prisma sets it on every update). Some log-like tables deliberately omit `updatedAt`.
Enums	Stored as plain `String` columns with documented value sets (roles, statuses, priorities). The TypeScript unions in `src/types/index.ts` mirror them.
JSON columns	**10 columns are native JSONB** (Prisma `Json`): `TestCase.steps/tags/attachments`, `TestCaseHistory.snapshot`, `ReviewEvent.flaggedFields`, `Bug.attachments`, `TaskReport.tasks`, `ActivityLog.beforeSnapshot/afterSnapshot/diff`. Array-typed ones carry `@default("[]")`.
Case sensitivity	Table names are quoted PascalCase (`"TestCase"`) — relevant when writing raw SQL in Neon/pgAdmin.
Datasource	`DATABASE_URL` (pooled, app traffic) + `DIRECT_URL` (schema tools). Identical locally.

Entity relationship overview



```
    ├── Notification
    └── ActivityLog (audit trail)
```

---

2. User

The account table. One row per person.

Column	Type	Default	Notes
`email`	String, **unique**	—	Login identifier, lowercase enforced by convention
`fullName`	String	—	Display name
`passwordHash`	String	—	bcrypt hash (10 rounds); never returned by any API
`role`	String	`"qa_engineer"`	`admin` | `team_lead` | `qa_engineer` | `viewer`
`telegramChatId`	String?	null	Reserved for future Telegram delivery (unused)
`avatarUrl`	String?	null	Optional avatar image URL
`isActive`	Boolean	`true`	`false` = soft-deleted user; login rejected
`notifyEmail`	Boolean	`true`	Preference flag (email sending not implemented)
`notifyTelegram`	Boolean	`true`	Preference flag (unused)
`notifyInapp`	Boolean	`true`	**Used**: if `false`, `createNotification()` skips in-app rows

Relations: this table is referenced by almost everything — modules owned/created/updated, module permissions, test cases authored/edited/assigned, review requests (as reviewer and as requester), review events, bugs reported/assigned, task reports, notifications, activity logs, refresh tokens. None of these cascade-delete the user; the user row is the anchor.

3. RefreshToken

Enables the "remember me for 7 days" experience with rotation.

Column	Type	Default	Notes
`token`	String, **unique**	—	The JWT refresh token itself
`userId`	String	—	→ User, **onDelete: Cascade**
`expiresAt`	DateTime	—	7 days after issue
`revoked`	Boolean	`false`	Set on refresh/logout (rotation)

Index: `userId`. Rotation story: every `POST /api/auth?path=refresh` revokes the old row and inserts a new one; a revoked token can never be reused.

4. Module

Feature area, arranged in a tree (root modules → submodules → …).

Column	Type	Default	Notes
`name`	String	—	e.g. `Authentication - Login`
`description`	String?	null	
`parentId`	String?	null	→ parent Module, **Cascade** (deleting a parent deletes children)
`path`	String	—	Materialised path of ancestor ids: `""` for roots, `"<rootId>"` for depth-1, `"<rootId>.<childId>"` for depth-2 …
`depth`	Int	`0`	0 = root
`ownerId`	String	—	→ User ("ModuleOwner"). Creator becomes owner automatically
`createdById`	String	—	→ User ("ModuleCreator"). **Immutable; governs delete rights**
`updatedById`	String?	null	→ User ("ModuleUpdater")

Indexes: `parentId`, `ownerId`. The `path`/`depth` pair makes "give me this module and all descendants" a prefix scan in JS (the API walks the tree once and matches by `path.startsWith`), and lets the UI indent the tree without extra queries.

5. ModulePermission

Per-module role override per user (schema supports it; the UI/API for granting is not wired yet — treated as reserved capability).

Column	Type	Notes
`moduleId` + `userId`	composite **unique**	One override per user per module
`roleOverride`	String	`team_lead` | `qa_engineer` | `viewer`
`grantedById`	String?	Which admin granted it

Both FKs **Cascade** — removing a module or user cleans its overrides.

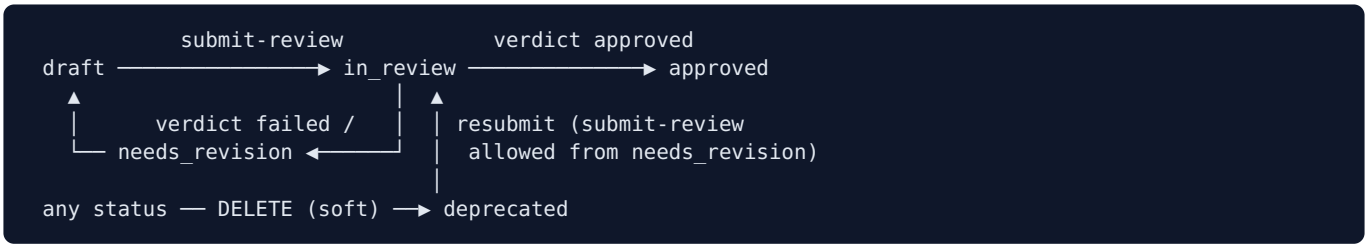
6. TestCase

The core entity.

Column	Type	Default	Notes
`moduleId`	String?	null	→ Module, **onDelete: SetNull** (deleting a module orphans its cases)
`title`	String	—	
`description`	String?	null	
`preconditions`	String?	null	
`steps`	**Json**	`[]`	Array of `{ stepNumber: number, action: string, expected: string }`
`expectedResult`	String?	null	
`priority`	String	`"P2"`	`P0` | `P1` | `P2` | `P3`
`status`	String	`"draft"`	`draft` | `in_review` | `needs_revision` | `approved` | `active` | `deprecated`
`tags`	**Json**	`[]`	Array of strings
`version`	Int	`1`	Bumped on every edit; old state stored in TestCaseHistory
`authorId`	String	—	→ User "TestCaseAuthor"; immutable
`lastEditedById`	String?	null	→ User "TestCaseEditor"
`assigneeId`	String?	null	→ User "TestCaseAssignee" — the suggested reviewer
`attachments`	**Json**	`[]`	Array (reserved for future file links)

Indexes: `moduleId`, `authorId`, `status` (status is the most-filtered column).

Status lifecycle



`active` exists as a post-approval "promoted" state (seed data uses it; the API does not transition into it yet).

7. TestCaseHistory

Immutable version snapshots, written **before** each update.

Column	Type	Notes
`testCaseId`	String	→ TestCase, **Cascade**
`version`	Int	The version that was current before the edit
`snapshot`	**Json** (required)	`{title, description, preconditions, steps, expectedResult, priority, tags, moduleId, assigneeId}`
`changedById`	String?	→ User
`changeSummary`	String?	`"Updated: <new title>"` or `"Field update"`

Index: `testCaseId`. Reading history = the UI's "Version History" card on the case detail page.

8. ModuleReviewRequest

Umbrella review for a whole module (bulk review workspace).

Column	Type	Default	Notes
`moduleId`	String	—	→ Module, **Cascade**
`reviewerId`	String	—	→ User "ModuleReviewer"
`requesterId`	String	—	→ User "ModuleReviewRequester"
`status`	String	`"pending"`	`pending` | `in_progress` | `completed`
`dueDate`	DateTime?	null	
`cycle`	Int	`1`	Per-module review round counter

Indexes: `reviewerId`, `moduleId`, `status`. Auto-transitions: `pending → in_progress` when the first child verdict lands; `→ completed` when **all** child case reviews are completed (server-side aggregation in `api/reviews` POST), or immediately via `approve-all`.

9. ReviewRequest

One review of one test case. Lives alone **or** as a child of a module review.

Column	Type	Default	Notes
`testCaseId`	String	—	→ TestCase, **Cascade**
`reviewerId`	String	—	→ User "Reviewer"
`requesterId`	String	—	→ User "ReviewRequester"
`status`	String	`"pending"`	`pending` | `in_progress` | `completed`
`dueDate`	DateTime?	null	
`cycle`	Int	`1`	Per-test-case round counter (resubmission increments)
`moduleReviewId`	String?	null	→ parent ModuleReviewRequest, **Cascade** — the bulk-review link

Indexes: `reviewerId`, `status`, `moduleReviewId`. Status transitions: `pending → in_progress` on a `needs_revision` verdict (the case needs rework, the review stays open); `→ completed` on `approved` / `failed`.

10. ReviewEvent

Append-only verdict history for a review request.

Column	Type	Notes
`reviewRequestId`	String	→ ReviewRequest, **Cascade**
`reviewerId`	String	→ User
`verdict`	String	`approved` | `failed` | `needs_revision`
`comment`	String?	**Required** by the API unless verdict is `approved`
`flaggedFields`	**Json**	`[]` default; array of human labels, e.g. `"Steps unclear"`

Index: `reviewRequestId`. The UI renders these as the "Review History" timeline with color-coded verdict badges.

11. Bug

Column	Type	Default	Notes
`title`	String	—	
`description`	String?	null	
`severity`	String	`"medium"`	`critical` | `high` | `medium` | `low`
`status`	String	`"open"`	`open` | `in_progress` | `resolved` | `closed` | `wont_fix`
`environment`	String?	null	e.g. `staging`, `production`
`stepsToReproduce`	String?	null	
`expectedBehaviour`	String?	null	
`actualBehaviour`	String?	null	
`reportedById`	String	—	→ User "BugReporter"
`assignedToId`	String?	null	→ User "BugAssignee"; assignment triggers a notification
`moduleId`	String?	null	→ Module (no explicit onDelete → DB default RESTRICT)
`attachments`	**Json**	`[]`	
`resolvedAt`	DateTime?	null	Server stamps `now()` when status becomes `resolved` / `closed`

Indexes: `status`, `severity`, `assignedToId`. Bugs are **hard-deleted** (admin only).

12. BugTestCaseLink

Many-to-many between bugs and test cases.

Column	Notes
`bugId` + `testCaseId`	Composite primary key (also the uniqueness)
Both FKs **Cascade**	Deleting a bug or case removes its links

Written at bug creation (`testCaseIds[]` body param) or via `PUT /api/bugs?id=…&action=link-tc`.

13. TaskReport

Daily work log — **one row per user per day**, enforced by `@@unique([reportedById, reportDate])`. The API uses find-then-update (upsert) on this.

Column	Type	Default	Notes
`reportedById`	String	—	→ User, **Cascade**
`reportDate`	DateTime	—	The day being reported
`tasks`	**Json**	`[]`	Array of `TaskItem` (below)
`notes`	String?	null	Free-text end-of-day notes
`totalMinutes`	Int	`0`	Server-computed sum of task minutes

`TaskItem` shape (inside the JSONB array): `{ description, timeSpentMinutes, moduleId | null, testCaseId | null, bugId | null, status: 'done' | 'in_progress' | 'blocked' }` — the three optional ids link a task line to any entity (rendered as pickers in the Tasks view).

Indexes: `reportedById`, `reportDate`.

14. Notification

In-app notification row (the bell).

Column	Type	Default	Notes
`userId`	String	—	Recipient, **Cascade**
`type`	String	—	`review_assigned` | `review_approved` | `review_failed` | `review_needs_revision` | `comment_added` | `bug_assigned` | `test_case_updated` | `module_created` | `daily_summary`
`title`	String	—	Short headline
`body`	String?	null	Longer text (e.g. includes verdict comment)
`entityType` / `entityId`	String?	null	Deep-link target: clicking navigates to the related view
`isRead`	Boolean	`false`	Flipped by PATCH / read-all

Index: composite `[userId, isRead]` — exactly the query the bell badge runs ("count unread for me").

15. ActivityLog

The audit trail. Written by `logActivity()` from **every** mutating route.

Column	Type	Notes
`userId`	String?	Actor; → User with **SetNull** (logs survive user deletion)
`action`	String	`CREATE` | `UPDATE` | `DELETE` | `STATUS_CHANGE` | `LOGIN` | `REVIEW_SUBMITTED` | `REVIEW_VERDICT` (some writers use suffixed forms like `REVIEW_SUBMITTED`)
`entityType`	String	`test_case` | `module` | `bug` | `review` | `user` | `task_report`
`entityId` / `entityName`	String?	What was touched; name keeps logs readable after renames
`beforeSnapshot`	**Json?**	Full relevant fields before the change
`afterSnapshot`	**Json?**	After the change
`diff`	**Json?**	`{ field: { before, after } }` — only changed keys; shallow JSON.stringify comparison
`ipAddress` / `userAgent`	String?	From `x-forwarded-for` / `user-agent` headers

Indexes: composite `[entityType, entityId]` (per-entity history card on the case detail page), `userId`, `action`. Activity rows are never deleted by the app.

---

16. Deletion semantics summary (important!)

Entity	Delete behaviour	Who may delete
Module	**Hard** — cascades to children, permissions, module reviews; test cases orphaned (`SetNull`)	Creator, team_lead, admin
Test case	**Soft** — `status → 'deprecated'`, row kept	Author, team_lead, admin
Bug	**Hard** — cascades links	**Admin only**
User	**Soft** — `isActive: false`; login blocked; data preserved	Admin (not on self)
TaskReport / Notification / ActivityLog	Not deletable via dedicated business rules (notifications deletable by owner; reports editable only by owner)	—

17. How JSONB is read back (the parser contract)

Prisma returns JSONB as already-parsed JavaScript objects. To stay compatible with legacy rows that might contain JSON strings, every read path funnels through:

```
parseJson(value, fallback)  // null → fallback; object → as-is; string → JSON.parse (safe)
toJson(value)               // pass-through, kept for call-site readability
```

Central normalisers in `src/lib/serializers.ts` (`serializeTestCase`, `serializeBug`, `serializeTaskReport`, `serializeActivityLog`, `serializeReviewEvent`, …) guarantee each JSON column is parsed **exactly once** before the row leaves the server — the class of bug that originally motivated them (`flaggedFields.map is not a function`) is documented in the project history.

03 · Authentication, Sessions & Permissions

> How a user proves who they are, how the app remembers them, and exactly who is allowed to do what. Server logic: `src/lib/auth.ts`; client logic: `src/lib/client/api.ts` + `src/store/index.ts`; UI: `src/components/auth-screen.tsx`.

---

1. The two tokens

	Access token	Refresh token
Format	JWT (HS256), payload `{ sub, email, role, fullName, jti }`	JWT with extra `nonce`, plus a random suffix for uniqueness
Lifetime	**15 minutes**	**7 days**
Stored where (client)	`localStorage['tcms-access']` **and** cookie `tcms-access` (path=/, max-age 7d, SameSite=Lax)	`localStorage['tcms-refresh']` only
Sent how	`Authorization: Bearer …` header on every API call; API routes also accept the cookie	JSON body of `POST /api/auth?path=refresh`
Server validation	`jsonwebtoken.verify` with `process.env.JWT_SECRET`	verify + lookup in `RefreshToken` table (must exist, not revoked, not expired)
Persisted server-side?	No (stateless)	Yes — one `RefreshToken` row enables revocation

Why both transports for the access token? The header is what `request<T>()` sends; the cookie is what Next.js route handlers read via `cookies()` in `getCurrentUser()`. Mirroring into the cookie happens in `setSession()` right after login/refresh.

2. Login flow (start to finish)

```
AuthScreen (login tab)
  → authApi.login({ email, password })          POST /api/auth?path=login
      Server:
        1. zod loginSchema { email, password(min 1) }
        2. db.user.findUnique({ email })         → 401 'Invalid credentials' if missing
           …or isActive === false                → 401 'Invalid credentials'
        3. bcrypt.compareSync(password, passwordHash) → 401 if wrong
        4. createSession(user):
             accessToken  = jwt.sign(payload, secret, { expiresIn:'15m' })
             refreshToken = jwt.sign(payload+nonce, secret, { expiresIn:'7d' })
             db.refreshToken.create({ userId, token: refreshToken, expiresAt: +7d })
        5. logActivity(LOGIN/user)
        6. → 200 { user (publicUser), accessToken, refreshToken }
  → setSession(access, refresh, user)   // localStorage ×3 + cookie
  → zustand setUser(user); toast 'Welcome back!'; AppShell renders
```

`publicUser` (the only user shape ever returned): `{ id, email, fullName, role, isActive, avatarUrl, telegramChatId, notifyEmail, notifyTelegram, notifyInapp, createdAt }` — **never** `passwordHash`.

Registration (`?path=register`) works identically with two extra rules: duplicate email → 400 `Email already registered`; and the **first user in an empty database is forced to `admin`** regardless of the requested role.

The register form offers only `qa_engineer | team_lead | viewer` (no admin option — admin comes from seeding or the first-user rule or an existing admin promoting someone).

3. Session restore & logout

Restore (page reload): `AuthGate` → `useAuthStore.initialize()` reads the stored user + tokens, then validates server-side with `GET /api/auth` (`me`). If the access token expired but the refresh token is still valid, the first `me` call gets a 401, `request<T>()` transparently refreshes (below) and retries — so a returning user within 7 days never sees a login screen. If validation fails outright → `clearSession()`.

Logout: `POST /api/auth?path=logout` with the refresh token in the body revokes the `RefreshToken` row (always 200), then the client wipes localStorage + cookie and the auth store. The 15-minute access token may technically still work until expiry — acceptable for this threat model; revocation protects the renewal path.

4. Automatic token refresh (the invisible 401 dance)

`request<T>(url, options, retry = true)` in `lib/client/api.ts`:

```
response.status === 401 && retry?
  → tryRefresh()  (single-flight: concurrent 401s share one refresh call)
      POST /api/auth?path=refresh { refreshToken }
      ok  → save new pair to localStorage → retry original request ONCE
      bad → clearSession() + window.location.reload()   // back to login screen
```

Server-side refresh (`?path=refresh`), in order: missing token → 400; row missing / `revoked` / expired → 401 `Invalid or expired refresh token`; JWT verify fail → 401; user missing/inactive → 401. On success the old row is revoked and a **new pair** is issued (rotation — a stolen refresh token stops working after the next legitimate refresh).

5. Roles

Four roles, hierarchy `viewer (0) < qa_engineer (1) < team_lead (2) < admin (3)`.

Role	Intended for	Gets
`viewer`	Read-only stakeholders	Browsing only: dashboard, module tree, case lists/details, notifications, profile
`qa_engineer`	Writing test cases	+ create cases, edit **own** cases, submit reviews, report bugs, daily task reports
`team_lead`	Reviewers/ managers	+ edit **any** case, perform reviews, create/edit modules, delete modules/cases, view all task reports, Reports view, Activity Log, bulk approve, see All Reviews tab
`admin`	System owner	+ user management (invite/role/deactivate), hard-delete bugs, act as reviewer on any review, everything team_lead has

Two enforcement layers exist and both are always applied:

1. **Permission matrix** (`lib/auth.ts` `can(role, action)`) — coarse capability check:

Permission	admin	team_lead	qa_engineer	viewer
create_module	✓	✓	—	—
edit_module	✓	✓	—	—
create_test_case	✓	✓	✓	—
edit_own_test_case	✓	✓	✓	—
edit_any_test_case	✓	✓	—	—
submit_review	✓	✓	✓	—
perform_review	✓	✓	—	—
create_bug	✓	✓	✓	—
assign_bug	✓	✓	—	—
create_task_report	✓	✓	✓	—
view_all_task_reports	✓	✓	—	—
view_activity_logs	✓	✓	—	—
manage_users	✓	—	—	—
grant_module_permissions	✓	✓	—	—

2. **Ownership checks** in the routes — e.g. a qa_engineer passes `edit_own_test_case` but the route additionally verifies `case.authorId === me.id`; review verdicts verify `review.reviewerId === me.id || role === 'admin'`.

The UI mirrors these rules so forbidden controls simply don't render (sidebar items, "New Test Case"/"Report Bug"/"Submit for Review" buttons, edit/delete buttons, inline role switches in Users). **The UI checks are convenience only — the API is the real gatekeeper.** A `viewer` calling the API directly gets 403.

6. Endpoint-by-endpoint protection map

Endpoint (method)	Auth	Permission / extra check
`POST /api/auth?path=register`	none	first user → admin
`POST /api/auth?path=login`	none	—
`POST /api/auth?path=refresh` / `logout`	none	refresh: valid unrevoked token row
`GET / PUT /api/auth` (me / profile / password)	✓	self only; role/email immutable via this route
`GET /api/seed` / `POST /api/seed`	none	no-op once any user exists
`GET /api/stats?action=dashboard`	✓	any role
`GET /api/stats?action=reports`	✓	`view_all_task_reports` (403 `Team lead or admin only`)
`GET /api/modules` (+ `?id=`)	✓	any role
`POST /api/modules`	✓	`create_module`
`PUT /api/modules?id=`	✓	`edit_module`
`PUT /api/modules?id=&action=submit-review`	✓	`submit_review`
`DELETE /api/modules?id=`	✓	`admin` / `team_lead` **or** module creator
`GET /api/test-cases` (list/history/reviews/detail)	✓	any role
`POST /api/test-cases`	✓	`create_test_case`
`PUT /api/test-cases?id=`	✓	`edit_any_test_case` **or** (author ∧ `edit_own_test_case`)
`PUT /api/test-cases?id=&action=submit-review`	✓	`submit_review`
`DELETE /api/test-cases?id=`	✓	author **or** `admin` / `team_lead` (soft delete)
`GET /api/reviews?id=` / `?action=queue`	✓	any role
`GET /api/reviews` (all, paginated)	✓	any role (weak spot — see §8)
`POST /api/reviews?id=` (verdict)	✓	`perform_review` ∧ (assigned reviewer ∨ admin)
`GET /api/module-reviews` (`?action=queue`)	✓	any role
`GET /api/module-reviews?action=all`	✓	`view_all_task_reports`
`GET /api/module-reviews?id=`	✓	any role
`POST /api/module-reviews?id=&action=approve-all`	✓	`perform_review` ∧ (assigned reviewer ∨ admin)
`GET /api/bugs` (list/detail/links)	✓	any role
`POST /api/bugs`	✓	`create_bug`
`PUT /api/bugs?id=` (fields)	✓	any authenticated user (by design: anyone can move a board card)
`PUT /api/bugs?id=` (with `assignedToId`)	✓	additionally `assign_bug`, else 403
`PUT /api/bugs?id=&action=link-tc`	✓	any authenticated user
`DELETE /api/bugs?id=`	✓	**admin only** (hard delete)
`GET /api/task-reports?action=my`	✓	self
`GET /api/task-reports?action=summary`	✓	`view_all_task_reports`
`GET /api/task-reports?id=`	✓	owner ∨ `view_all_task_reports`

Endpoint (method)	Auth	Permission / extra check
`GET /api/task-reports` (list)	✓	non-managers force-filtered to own rows
`POST /api/task-reports` (upsert)	✓	`create_task_report`; row owned by caller
`PUT /api/task-reports?id=`	✓	**owner only** (even admins can't edit others' reports)
`GET /api/activity-logs` (full list)	✓	`view_activity_logs`
`GET /api/activity-logs?action=entity` / `=user`	✓	any authenticated user (check happens after these branches — see §8)
`GET /api/notifications`	✓	always scoped to `userId = me`
`POST /api/notifications?action=read-all`	✓	self rows only
`PATCH/DELETE /api/notifications?id=`	✓	owner only (400 `Not your notification` otherwise)
`/api/users` **all methods**	✓	`manage_users` → effectively **admin only** (403 `Admin only`)

7. Password change

`PUT /api/auth?path=password` with `{ currentPassword, newPassword }`: requires min 8 chars, verifies the current password (`400 Current password is incorrect` on mismatch), re-bcrypts and saves, writes an `UPDATE/user` activity log. The client additionally double-checks the confirmation field and length before even sending.

8. Honest security notes (worth knowing)

1. `JWT_SECRET` **fallback**: if the env var is missing, a hardcoded dev secret is used. The deployment guide's checklist makes setting it mandatory — on Vercel it is one of the 3 required variables.
2. **Weaker branches** (documented, by design acceptable for a team tool): `GET /api/reviews` (all) has no role gate; `activity-logs?action=entity|user` run before the permission check. Both still require a valid login.
3. **WS mini-service does not validate tokens** (`token: 'dev'` placeholder) — it only relays notification payloads to a `user:<id>` room; compromise impact is limited to seeing your own notification stream early. Documented in `12-realtime-and-deployment.md`.
4. No CSRF protection beyond `SameSite=Lax` cookie + Bearer-header requirement — the API never relies on the cookie alone (a browser-initiated cross-site form post would lack the `Authorization` header).
5. Search filters use case-sensitive `contains` on PostgreSQL — a correctness quirk, not a security one (documented in `04-api-reference.md`).

04 · API Reference — every endpoint

> All endpoints live under `/api/*` in `src/app/api/**/route.ts`. Conventions: **query-param addressing** (`?id=`, `?action=`, auth also accepts `?path=`), JSON bodies, errors as `{ "error": "message" }` with proper status codes (400/401/403/404), list responses via `paginate() = { data, page, pageSize, total, totalPages }`. Unless stated otherwise every endpoint requires the `tcms-access` JWT (401 `Unauthorized` otherwise).
>
> Legend: 🔓 no auth · 🛡 permission gate · 👑 ownership/extra check

> ⚠️ **SEPTEMBER 2026 RESTRUCTURE:** every API resource now uses the layered architecture Controller (`route.ts`) → Validation (`*.validation.ts`) → Service (`*.service.ts`) → Repository (`*.repository.ts`, raw SQL only), with responses through `src/lib/api.ts` (`ok`, `paginate`, `ApiError`, `withApi`) and structured logging (`src/lib/logger.ts`). Entity keys in responses are renamed end-to-end (`id` → `userId`/`moduleId`/`testCaseId`/`bugId`/...). Unknown actions now consistently return 400; search is case-insensitive (`ILIKE`). Details: `REFACTORING-REPORT.md`.

---

1. `GET /api` 🔓

Health check. Returns `{ "message": "Hello, world!" }`. No DB.

2. `/api/auth` — authentication (`src/app/api/auth/route.ts`)

Action dispatch: `?path=register | login | refresh | logout` (falls back to an `x-action` header, unused by the client). Unknown action → 400 `Unknown action`.

Endpoint	Auth	Request	Success response
`POST ?path=register`	🔓	`{ email, fullName(min 2), password(min 8), role? }`	`{ user, accessToken, refreshToken }`
`POST ?path=login`	🔓	`{ email, password }`	same
`POST ?path=refresh`	🔓	`{ refreshToken }`	`{ accessToken, refreshToken }` (rotated)
`POST ?path=logout`	🔓	`{ refreshToken }`	`{ "success": true }`
`GET /api/auth`	✓	—	`publicUser` (validated against DB)
`PUT /api/auth`	✓	profile fields **or** `?path=password`	`publicUser` / `{ "success": true }`

Business rules:

- **register**: duplicate email → 400 `Email already registered`; **first user in an empty DB is forced to `admin`**; otherwise role defaults to `qa_engineer`.
- **login**: unknown email, deactivated user or wrong password all return the same 401 `Invalid credentials` (no user enumeration).
- **refresh**: validates the `RefreshToken` row (exists ∧ not revoked ∧ not expired ∧ JWT valid ∧ user active), revokes it, issues a fresh pair.

- **PUT (profile)**: only `fullName, avatarUrl, telegramChatId, notifyEmail, notifyTelegram, notifyInapp` are accepted — role/email are immutable here.
- **PUT ?path=password**: verifies `currentPassword`, requires `newPassword ≥ 8`.

Side effects: `CREATE/user` or `UPDATE/user` activity logs; `LOGIN` log on login.

3. `/api/seed` — demo data 🔓 (`seed/route.ts`)

`GET` (added for browser seeding) and `POST` behave identically. **Idempotent**: if `user.count() > 0` → `{ seeded: false, message: "Database already has users; skipping seed." }`. Otherwise inserts exactly: 4 users (admin/teamlead/qa1/qa2 with the documented passwords), 9 modules (3 roots × login+validation children), 5 test cases (one in each key status), 3 bugs, 2 task reports. Returns `{ seeded: true, users: […4 emails], modules: 9, testCases: 5, bugs: 3, taskReports: 2 }`. No activity logs / notifications are produced by the seed.

4. `/api/stats` — aggregates (`stats/route.ts`) 🛡 any role

Endpoint	Returns
`GET ?action=dashboard`	`{ totalTestCases, statusBreakdown[{status,count}], openBugs, severityBreakdown[{severity,count}], myPendingReviews, recentActivity(10, with user) }`
`GET ?action=reports` 🛡 `view_all_task_reports`	`{ testCasesByStatus, testCasesByPriority, bugsByStatus, weeklyHours[{user,week,minutes,hours}] }` — last 30 days, Sunday-based weeks

`openBugs` counts `status ∈ ['open','in_progress']`; `myPendingReviews` counts reviews where I am reviewer and status ∈ pending/in_progress. Quirk: an unknown action here returns **200** `{ error: 'Unknown action' }` (elsewhere it's 400).

5. `/api/modules` — module tree (`modules/route.ts`)

Endpoint	Auth rules	Behaviour
`GET`	any role	`{ tree, flat }` — every module with `owner`, `testCaseCount` (groupBy), nested `children`
`GET ?id=`	any role	`{ module (incl. owner/createdBy/updatedBy/permissions), testCases (desc by updated), submodules }`; 404 `Module not found`
`POST` 🛡 `create_module`	body `{ name(min 1), description?, parentId? }`	Root: `path=''`, depth 0. Child: parent 404-checked; `path=parent.path + '.' + parentId`, `depth+1`. **`ownerId` is never taken from the client — owner = creator.** Logs `CREATE/module`
`PUT ?id=` 🛡 `edit_module`	body `{ name?, description?, ownerId? }`	Ownership transfer allowed; `createdById` immutable (governs delete rights). Sets `updatedById`. Logs `UPDATE/module` with before/after
`PUT ?id=&action=submit-review` 🛡 `submit_review`	body `{ reviewerId?, dueDate?, includeSubmodules=true }`	**Module-level review** — see below
`DELETE ?id=` 🛡	`admin` / `team_lead` **or** creator; else 403 `Only the creator of this module or a senior manager (team lead / admin) can delete it.`	**Hard delete**; DB cascades children/permissions/module reviews; cases get `moduleId=null`. Logs `DELETE/module`

`action=submit-review` step by step

1. Resolve reviewer: body `reviewerId` → else first **active** user with role `team_lead` / `admin` → none: 400 `No reviewer available. Please assign a team lead.`
2. Target set = module (+ all descendants when `includeSubmodules` — iterative stack walk).
3. Eligible cases = in targets ∧ `status ∈ ['draft','needs_revision']`; none → 400 `No eligible test cases (draft or needs_revision) found in this module.`
4. `cycle = max(existing module cycles) + 1` → create `ModuleReviewRequest(pending)`; `createMany` child `ReviewRequest` per case (same cycle, `moduleReviewId` set); `updateMany` cases → **`in_review`**.
5. Log `REVIEW_SUBMITTED/module`; notify reviewer (`review_assigned`, entityType `module_review`). Returns `{ moduleReview, caseReviewCount, testCaseIds }`.

6. `/api/test-cases` — the core domain (`test-cases/route.ts`)

Endpoint	Auth rules	Behaviour
`GET` list 🛡 any	params `page(1)`, `pageSize(20)`, `moduleId`, `status`, `priority`, `authorId`, `search`	`paginate` of serialized cases, fixed sort `updatedAt desc`. `search` = title `contains` (**case-sensitive** on PostgreSQL)
`GET ?id=`	any	single serialized case; 404 `Test case not found`
`GET ?id=&action=history`	any	`TestCaseHistory[]` (version desc, with `changedBy`) — snapshots parsed
`GET ?id=&action=reviews`	any	`ReviewRequest[]` for the case (with reviewer/requester/events)
`POST` 🛡 `create_test_case`	body `{ moduleId?, title(min 1), description?, preconditions?, steps[] (default []), expectedResult?, priority (default P2), tags[] (default []), assigneeId? }`	Creates with `status:'draft'`, `authorId = me`, `version 1`. Logs `CREATE/test_case`
`PUT ?id=` 🛡	`edit_any_test_case` ∨ (author ∧ `edit_own_test_case`); else 403 `You can only edit your own test cases`	**Versioned update** — see below
`PUT ?id=&action=submit-review` 🛡 `submit_review`	body `{ reviewerId?, dueDate? }`	**Single-case review** — see below
`DELETE ?id=` 🛡	author ∨ admin/team_lead	**Soft delete**: `status → 'deprecated'`. Logs `DELETE/test_case` with before/after status

Regular PUT (versioned update) — the heart of case editing

1. Fetch case (404 first), then permission check (above).
2. **Snapshot the current state** into `TestCaseHistory` (`version = tc.version`, `changedById = me`, summary `"Updated: <new title>"` if the title changes, else `"Field update"`).
3. Update with `version: tc.version + 1`, `lastEditedById = me`, partial fields.
4. **Status is never touched by an edit** (approved stays approved).
5. Logs `UPDATE/test_case` with full serialized before/after (diff computed server-side).
6. If `assigneeId` changed → notify new assignee (`test_case_updated`).

`action=submit-review`

Reviewer resolution: body `reviewerId` → case `assigneeId` → first active team_lead/admin (400 `No reviewer available. Assign a reviewer first.`). `cycle = max(case review cycles) + 1` → create `ReviewRequest(pending)`; case → **`in_review`** (version NOT bumped). Logs `REVIEW_SUBMITTED/test_case`; notifies reviewer (`review_assigned`, entityType `review`). Returns the review row.

7. `/api/reviews` — single-case reviews (`reviews/route.ts`)

Endpoint	Auth rules	Behaviour
`GET ?action=queue` 🛡 any	—	My open queue (`reviewerId = me`, status pending/in_progress), with testCase/author/module summary
`GET ?id=`	any	Full detail: review + testCase + reviewer + requester + events
`GET` (list)	any role	All reviews, paginated (pageSize 50), filters `status`, `reviewerId`, `testCaseId` — no permission gate on this branch (see 03 §8)
`POST ?id=` 🛡 `perform_review` 👑	assigned reviewer ∨ admin; else 403 `You are not the assigned reviewer`	**Verdict** — see below

Verdict (`POST ?id=`)

Body: `{ verdict: 'approved'|'failed'|'needs_revision', comment?, flaggedFields[] }`. Rule: **comment required unless approved** → 400 `Comment is required when verdict is not "approved"`.

1. Create `ReviewEvent { verdict, comment, flaggedFields }`.
2. ReviewRequest status: `needs_revision` → `in_progress` (stays open for the rework loop); `approved` / `failed` → `completed`.
3. TestCase status: `approved` → `approved`; `failed` **or** `needs_revision` → `needs_revision`.
4. Log `REVIEW_VERDICT/test_case` (before: old case status; after: new status + verdict).
5. Notify the **author** (`review_approved` / `review_failed` / `review_needs_revision`), body includes the verdict comment.
6. **Module aggregation**: if this review has `moduleReviewId` → check all siblings; every sibling `completed` ⇒ parent module review → `completed` + notify its requester (`review_approved`, title `Module review completed`); otherwise parent → `in_progress`.
7. Returns `{ event, reviewStatus, testCaseStatus, moduleReviewCompleted? }`.

8. `/api/module-reviews` — bulk reviews (`module-reviews/route.ts`)

Endpoint	Auth rules	Behaviour
`GET ?action=queue`	any	Mine, pending/in_progress, with `module` + `_count.caseReviews`
`GET ?action=all` 🛡 `view_all_task_reports`	lead/admin	Latest 100 with module/reviewer/requester + counts; else 403 `Team lead or admin only`
`GET ?id=`	any	Detail: module, reviewer, requester, **caseReviews[]** each embedding full serialized testCase + events
`POST ?id=&action=approve-all` 🛡 `perform_review` 👑	assigned ∨ admin	**Bulk approve** — see below

Bulk approve

Body `{ comment? }` (default `"Bulk approved with module review."`). Pending set = case reviews not `completed`; empty → 400 `No pending case reviews to approve.` For each: `ReviewEvent(approved, comment)` → review `completed` → its case → **`approved`**. Parent module review → `completed`. Logs `REVIEW_VERDICT/module_review`; notifies requester (`review_approved`, title `Module review approved`). Returns `{ approvedCount, moduleReviewStatus: 'completed' }`.

9. `/api/bugs` — bug tracker (`bugs/route.ts`)

Endpoint	Auth rules	Behaviour
`GET` list	any	`page(1)`, `pageSize(100)`, filters `status`, `severity`, `moduleId`, `assignedToId`, `reportedById`, `search` (title contains, case-sensitive); sort `createdAt desc`
`GET ?id=`	any	Bug + module + reportedBy + assignedTo + testCaseLinks
`GET ?id=&action=links`	any	`BugTestCaseLink[]` with testCase summaries
`POST` 🛡 `create_bug`	body `{ title, description?, severity(default medium), environment?, stepsToReproduce?, expectedBehaviour?, actualBehaviour?, moduleId?, assignedToId?, testCaseIds[] }`	`reportedById = me`; optional `createMany` links. Logs `CREATE/bug`; if assigned → notify assignee (`bug_assigned`)
`PUT ?id=`	any authenticated user for fields; **`assign_bug` required if payload contains `assignedToId`** (403 `Only team leads and admins can reassign bugs`)	Partial update. Status `resolved`/`closed` ⇒ server stamps `resolvedAt = now()`. Logs `UPDATE/bug` (full before/after); assignee change → `bug_assigned` notification
`PUT ?id=&action=link-tc`	any	Body `{ testCaseId }` → create link (duplicate-safe); always `{ "success": true }`
`DELETE ?id=` 👑	**admin only** (403 `Admin only`)	**Hard delete** (links cascade). Logs `DELETE/bug`

The deliberately open field-edit rule is what lets any teammate drag a card across the Kanban board without role friction; only assignment and deletion are gated.

10. `/api/task-reports` — daily work log (`task-reports/route.ts`)

Endpoint	Auth rules	Behaviour
`GET ?action=my`	self	All my reports (desc by date), serialized
`GET ?action=summary` 🛡 `view_all_task_reports`	lead/admin	Per-user aggregates for a date range (default last 30 days): `{ users: [{ userId, userName, totalMinutes, days, hours }] }`
`GET ?id=` 👑	owner ∨ `view_all_task_reports`	Single report; else 403 `You can only view your own reports`
`GET` list	any	**Non-managers force-filtered to own rows**; managers may pass `userId`; filters `dateFrom`, `dateTo`, `moduleId` (substring match inside the tasks JSON — heuristic)
`POST` 🛡 `create_task_report`	body `{ reportDate, tasks[] (default []), notes? }`	**Upsert by (me, reportDate)** — unique constraint; recomputes `totalMinutes`. Logs `CREATE` or `UPDATE/task_report`
`PUT ?id=` 👑	**owner only** — even admins	Partial update, recomputes total. Logs `UPDATE/task_report`

`tasks[]` item validation: `{ description(min 1), timeSpentMinutes(int ≥0), moduleId?, testCaseId?, bugId?, status: done|in_progress|blocked }`.

11. `/api/activity-logs` — audit trail (`activity-logs/route.ts`)

Endpoint	Auth rules	Behaviour
`GET` full list 🛡 `view_activity_logs`	lead/admin	`page(1)`, `pageSize(50)`, filters `userId`, `entityType`, `action`, `dateFrom`, `dateTo`; desc; snapshots/diff parsed
`GET ?action=entity&type=&id=`	any authenticated (checked after this branch)	Latest 50 logs for one entity (used by the case-detail Activity card)
`GET ?action=user&id=`	any authenticated (same)	Latest 50 logs by one user

12. `/api/notifications` — the bell (`notifications/route.ts`)

Everything is scoped to the logged-in user.

Endpoint	Behaviour
`GET`	My notifications, paginated (pageSize 50), optional `isRead=true
`POST ?action=read-all`	`updateMany` my unread → read; `{ "success": true }`
`PATCH ?id=`	Mark one read; owner check → 400 `Not your notification`
`DELETE ?id=`	Hard delete one of my notifications

13. `/api/users` — admin user management 👑 (`users/route.ts`)

Every method requires `manage_users` → admin only (403 `Admin only`).

Endpoint	Behaviour
`GET`	Paginated `publicUser`s (pageSize 50), filters `role`, `isActive`; sort `createdAt asc`
`POST`	Invite: `{ email, fullName(min 2), password(min 8), role (required) }`; duplicate email → 400. Logs `CREATE/user`
`PUT ?id=`	`{ fullName?, role?, isActive?, avatarUrl? }`. Logs `UPDATE/user` (before/after role+active+name)
`DELETE ?id=`	**Soft delete**: `isActive: false`; self-delete → 400 `Cannot delete yourself`. Logs `DELETE/user`

14. `/api/keep-alive` — scheduled anti-sleep ping (`keep-alive/route.ts`) 🔓

Public by design — Vercel Cron sends unauthenticated GETs, and the endpoint leaks nothing. `GET` runs `SELECT 1` through Prisma and answers `{ ok: true, message: …, pingedAt }`; on database failure 500 `{ ok: false }`. Purpose: free database providers pause idle projects (Supabase after 7 days of no activity). `vercel.json` schedules a daily cron — the Hobby plan's maximum frequency — to call this endpoint, so the database records activity every day and never pauses. Harmless with every provider; also callable manually from a browser. See doc 12 §3 and DEPLOYMENT-GUIDE.md Appendix E.

15. Response envelope examples

```
// GET /api/test-cases?status=draft&page=1
{ "data": [ { "id": "clx…", "title": "…", "steps": [ { "stepNumber": 1, "action": "…", "expected":
"…" } ],
```

```
    "tags": ["smoke"], "status": "draft", "priority": "P2", "version": 3,
    "module": { "id": "…", "name": "Authentication - Login" }, "author": { "id": "…", "fullName": "Quinn
Tester" },
    "lastEditedBy": null, "assignee": null } ],
  "page": 1, "pageSize": 20, "total": 1, "totalPages": 1 }

// POST /api/reviews?id=… (verdict)
{ "event": { "id": "…", "verdict": "needs_revision", "comment": "…", "flaggedFields": ["Steps
unclear"] },
  "reviewStatus": "in_progress", "testCaseStatus": "needs_revision" }
```

16. Known quirks (documented on purpose)

1. `stats` unknown action → 200 with `{error}` (everywhere else 400).
2. `GET /api/reviews` (all) and `activity-logs ?action=entity|user` have weaker gates (see `03-authentication-and-permissions.md` §8).
3. Notification ownership violations return **400** rather than 403.
4. `search` filters are case-sensitive on PostgreSQL (Prisma `contains` without `mode: 'insensitive'`).
5. Task-report date uniqueness is on the stored datetime — the client normalises dates to `YYYY-MM-DD` (timezone-safe `todayISO()`), so in practice one row per calendar day.
6. The Reports view's date inputs are currently cosmetic (not sent to the API).

05 · Frontend Architecture — how the client is built

> The browser side is a single-page application: one URL, fourteen views, three state systems with clear jobs. This document explains the machinery; documents 06–10 walk through each view.

---

1. Three state systems, three jobs

System	File	Owns	Example
Zustand (client state)	`src/store/index.ts`	Who am I, where am I, what did the bell just tell me	`useAuthStore`, `useViewStore`, `useNotificationsStore`
React Query (server state)	`providers.tsx` + usage in views	What has the API returned, when is it stale, what to refetch after a mutation	`['test-cases','list',filters]`
Component state (`useState`)	each view	Transient UI: dialogs, text inputs, page numbers	`createOpen`, `search`, `page`

Rule of thumb: **nothing derived from the server is ever copied into Zustand** (except notifications, which are push-updated); Zustand's `user` is a mirror of the server session used for instant rendering and permission gating.

2. Zustand stores (exact shape)

`useAuthStore`

```
{ user, isLoading, isInitialized, booted,
  initialize()                    // read localStorage → GET /api/auth → validate
  login(email, password)          // authApi.login → setSession → setUser
  register(data)                  // same for register
  logout()                        // authApi.logout → clearSession → user=null
  setUser(user)                   // used by profile updates }
```

`initialize()` runs once from `AuthGate`; until `booted && isInitialized` the whole app shows a centred spinner. This is the **session restore** path.

`useViewStore` — the router replacement

```
type ViewKey = 'dashboard' | 'modules' | 'test-cases' | 'reviews' | 'bugs' |
  'tasks' | 'reports' | 'activity-log' | 'notifications' | 'settings-users' |
  'settings-profile' | 'test-case-detail' | 'review-detail' | 'module-review-detail'

{ current: ViewKey, params: Record<string,string>,
  navigate(view, params?) }       // plain set(); params used for { id } deep links
```

`AppShell` maps `current` → component with a `switch`; the three `*-detail` views read `params.id`. There is **no URL routing** — refresh always lands on the dashboard (the session, not the view, is restored).

`useNotificationsStore`

```
{ items: Notification[], unreadCount,
  setItems(list)                  // seeded from GET /api/notifications (50)
  addNotification(n)              // WS push → prepend, cap 50
  markAllRead() / markRead(id) / remove(id) }
```

Consumed by the topbar bell, the Notifications view, and the socket hook.

3. The data layer — `src/lib/client/api.ts`

A plain fetch client (no React Query code in it). Pieces:

1. **Session helpers** — `getAccessToken/getRefreshToken/getStoredUser/setSession/clearSession` over `localStorage` keys `tcms-access`, `tcms-refresh`, `tcms-user`; `setSession` also writes the `tcms-access` cookie so server routes can read the JWT.
2. **`request<T>(url, options, retry=true)`** — sets `Content-Type: application/json` and `Authorization: Bearer <token>`; on 401 (first attempt) calls `tryRefresh()` (single-flight) then retries once; on refresh failure wipes the session and reloads. Non-OK responses throw `ApiError { status, data, message }` where `message` comes from the server's `{ error }` body. 204 → `undefined`.
3. **Namespaced endpoint functions** — `authApi`, `usersApi`, `modulesApi`, `testCasesApi`, `reviewsApi`, `moduleReviewsApi`, `bugsApi`, `taskReportsApi`, `notificationsApi`, `activityLogsApi`, `statsApi`, `seedApi`. One function per endpoint; they only build URLs and parse nothing. (Full inventory with routes: `04-api-reference.md`.)

4. React Query conventions (live in the views)

Global defaults (`providers.tsx`): **staleTime 30 s · retry 1 · refetchOnWindowFocus false**. Two polling loops exist: dashboard stats and the two sidebar review queues (`refetchInterval: 30_000`).

Cache key inventory (exact)

Key	Used by	Notes
`['stats','dashboard']`	dashboard	30 s polling
`['stats','reports']`	reports	
`['modules','tree']`	modules + every module picker	
`['modules','detail',id]`	modules view	`enabled: !!selectedId`
`['test-cases','list', {search,statusFilter,priorityFilter,moduleFilter,page}]`	test-cases table	filters are part of the key
`['test-cases','list','all-light']`	tasks/bugs pickers	pageSize 200
`['test-cases','detail',id]` / `['test-cases','history',id]` / `['test-cases','reviews',id]`	case detail page	
`['reviews','queue']` + `['module-reviews','queue']`	sidebar badge + reviews view	30 s polling, `enabled: !!user`
`['reviews','all']` / `['module-reviews','all']`	reviews "All" tab	`enabled: tab==='all'`; module-all is lead/admin gated
`['reviews','detail',id]` / `['module-reviews','detail',id]`	detail pages	
`['bugs','list']`	bugs board/table	pageSize 200
`['users','all']` / `['users','list',params]`	pickers, Users admin, activity filter	
`['task-reports','my']` / `['task-reports','user',userId,date]`	tasks view	manager vs self variants
`['notifications']`	— (store is primary)	invalidated after read/ delete
`['activity-logs',params]` / `['activity','test_case',id]`	activity log, case detail	

Mutation pattern (no optimistic updates, by design)

```
mutation.mutate(body)
  onSuccess:  toast.success('…')
              qc.invalidateQueries({ queryKey: [rootKey, …] })   // sometimes precise keys too
              navigate(…) when a redirect makes sense
  onError:    handleApiError(err, 'Fallback message')
```

Coarse root keys (`['test-cases']`, `['modules']`, `['bugs']`, `['reviews']`, `['module-reviews']`, `['users']`, `['task-reports']`, `['notifications']`) keep invalidation simple and correct; precise keys (`['reviews','queue']`, `['test-cases','detail',id]`) refresh the visibly-open data faster. The cost of this simplicity — a few extra refetches — is negligible at this data scale.

5. Error handling & toasts

- **sonner** is the toast engine (`import { toast } from 'sonner'`); `layout.tsx` mounts `<SonnerToaster position="top-right"/>` (a legacy radix `<Toaster/>` is also mounted but unused by views).

- `lib/errors.ts` maps thrown `ApiError`s to friendly text: network → "Network error — please check your connection."; 400 → "Invalid input…"; 401 → "Your session has expired…"; 403 → "You do not have permission to do this."; 404 → "This item no longer exists."; 500 → "Server error…"; server-provided `data.error` always wins over the generic text.
- `handleMutation(promise, { success })` wraps the common then/catch/toast dance.

6. Rendering building blocks

- **shadcn/ui primitives** (`src/components/ui/*`, ~40 files): Radix-based, Tailwind-styled, template-provided. Views mainly use button, card, dialog, alert-dialog, select, input, textarea, table, badge, tabs, switch, checkbox, sheet, popover, dropdown-menu, avatar, tooltip, scroll-area, calendar, alert, separator.
- **shared/state.tsx** — one vocabulary for async UI: `StateLoading(rows)`, `StateLoadingCards(n)`, `StateSpinner(label)`, `StateEmpty({icon,title,…,action})`, `StateError({message,onRetry})`.
- **shared/pagination.tsx** — `<Pagination page totalPages total onPageChange>`; renders nothing when there's a single page. Used by the test-cases table; activity-log and users use a custom Prev/Next pair because they disable buttons while `isFetching`.
- **lib/format.ts** — every color map and formatter the views use (exact mappings): `statusColor` (draft slate, in_review amber, needs_revision orange, approved emerald, active green, deprecated zinc), `priorityColor` (P0 red, P1 orange, P2 yellow, P3 sky), `bugStatusColor`, `severityColor`, `verdictColor`, `roleColor` (admin purple, team_lead indigo, qa teal, viewer slate), `actionColor` (by prefix: CREATE emerald, UPDATE amber, DELETE rose, STATUS sky, LOGIN slate, REVIEW purple), `reviewStatusColor`; formatters `formatDateTime`, `formatDate`, `timeAgo`, `minutesToHuman`, `truncate` (all `en-US`).

7. Component composition of a typical view

Every view follows the same skeleton — this consistency is what makes the app easy to extend:

```
export default function SomethingView() {
  const { user } = useAuthStore()               // 1. identity + role gates
  const qc = useQueryClient()
  const { data, isLoading } = useQuery(key, fn) // 2. server data
  const [filter, setFilter] = useState(...)     // 3. local UI state
  const x = useMutation(...)                    // 4. mutations w/ toast+invalidate
  if (isLoading) return <StateLoading/>         // 5. async states
  return (                                       // 6. layout: header → filters → content
    <div>…toolbar… table/cards… dialogs…</div>
  )
}
```

Role gating is inline (`user.role === 'admin' || user.role === 'team_lead'`) rather than via a helper — visible, greppable, and mirrored by the API's real checks.

8. Theming & styling

- `globals.css` (Tailwind v4): `@theme inline` maps shadcn CSS variables (`--background`, `--primary`, `--chart-1..5`, `--sidebar-*`, `--radius: 0.625rem`) in **oklch**, with a `.dark` class variant. `tailwind.config.ts` keeps the legacy hsl-var mapping for compatibility.
- Fonts: Geist / Geist Mono via `layout.tsx`.
- Charts use hardcoded hex palettes (documented per view in docs 06/09).

9. Quirks worth knowing (documented, harmless)

1. `use-toast.ts` + radix `<Toaster/>` are mounted but unused (views use sonner).
2. `use-mobile.ts` exists; views rely on CSS breakpoints (`hidden md:flex` + drawer) instead.

3. Module-review per-case Fail/Needs-Revision prompts use `window.prompt` while the single review detail uses a rich dialog — a UX inconsistency, not a bug.
4. Bug delete inside the detail sheet has **no confirmation dialog**.
5. The socket hook's WS auth sends `token: 'dev'` (server doesn't validate — see doc 12).

06 · Pages — Login / Register & Dashboard

> Screens: **AuthScreen** (`src/components/auth-screen.tsx`) and **Dashboard** (`src/components/views/dashboard.tsx`). These are the first two things every user sees; together they illustrate the app's whole pattern: gate → fetch → render → navigate.

---

1. AuthScreen — login & register

When it shows

`AppShell` checks `useAuthStore().user`; if there is no logged-in user the shell is never rendered — the **entire screen** is the auth experience. The boot spinner in `AuthGate` guarantees this decision is made only after the stored session was validated (or cleared).

Layout

A split screen: left panel = product pitch (title, feature bullets, gradient accent); right panel = a card with two tabs, **"Sign in"** and **"Register"**, plus the demo account quick-fill buttons.

Sign-in tab (state: `mode='login'`)

Element	Behaviour
Email, Password inputs	plain controlled state (`email` , `password`)
"Sign in" button	`authStore.login(email, password)` → `POST /api/auth?path=login`
success	`setSession(...)` , toast `Welcome back!` , `AppShell` re-renders as the app
failure	toast `err.message` — typically Invalid credentials (401)

Register tab (`mode='register'`)

Element	Behaviour
Full name / Email / Password ("Min. 8 characters") / Role select	state `fullName, email, password, role`
Role select options	`qa_engineer` (default), `team_lead` , `viewer` — **no admin option** (first-user-in-empty-DB rule or admin promotion only)
"Create account" button	`authStore.register({ email, fullName, password, role })` → `POST /api/auth?path=register`
success	toast `Account created` ; user lands directly in the app

Demo accounts card

Four one-click buttons fill the form: `admin@tcms.com / Admin@123` , `teamlead@tcms.com / Lead@123` , `qa1@tcms.com / QA@12345` , `qa2@tcms.com / QA@12345` . They exist because the `/api/seed` endpoint creates exactly these users (documented in the deployment guide).

Flow to remember

Login does **not** navigate anywhere — the auth store flips `user`, and the same `page.tsx` now renders the shell whose default view is `dashboard`.

---

2. Dashboard — the landing overview

File: `src/components/views/dashboard.tsx` (236 lines). **ViewKey:** `'dashboard'` (default after login). **Visible to:** every role.

Data

One query — `['stats','dashboard']` → `GET /api/stats?action=dashboard`, polling every 30 s:

```
{ totalTestCases, statusBreakdown[{status,count}], openBugs,
  severityBreakdown[{severity,count}], myPendingReviews, recentActivity[10] }
```

`myPendingReviews` is personal (reviews where I am the reviewer), which is why the same dashboard shows different numbers to different roles.

Screen anatomy (top → bottom)

1. **Greeting header** — `Welcome back, {first name}` 👋 from the auth store, with two quick actions on the right: **"New Test Case"** → `navigate('test-cases')` and **"Report Bug"** → `navigate('bugs')`.
2. **Four KPI cards** (clickable, each navigates):

Card	Value source	Navigates to
Total Test Cases	`totalTestCases`	`test-cases`
Open Bugs	`openBugs`	`bugs`
My Pending Reviews	`myPendingReviews`	`reviews`
Approved Test Cases	`statusBreakdown` entry `approved`	`test-cases`

3. **Two charts (recharts), side by side:**
   - Test Cases by Status — donut `PieChart` over `statusBreakdown`. Colors: draft `#64748b`, in_review `#f59e0b`, needs_revision `#f97316`, approved `#10b981`, active `#22c55e`, deprecated `#a3a3a3`.
   - Open Bugs by Severity — horizontal `BarChart` over `severityBreakdown`. Colors: critical `#dc2626`, high `#ea580c`, medium `#f59e0b`, low `#0ea5e9`.
4. **Recent Activity feed** — last 10 `ActivityLog` rows (action badge, actor avatar, entity name, `timeAgo`); **"View all"** → `navigate('activity-log')` (menu item itself is lead/admin only, but the dashboard feed renders for everyone).

States & permissions

- Loading: skeleton grid (`StateLoadingCards`).
- Query error: an `AlertTriangle` card "Failed to load dashboard data."
- Empty activity: "No recent activity".
- No mutations, no dialogs, no role gates — the dashboard is read-only by design.

Why it matters architecturally

The dashboard is the only screen that aggregates across all domains in a single request — the `/api/stats` route runs six Prisma queries in parallel and returns one shaped object, deliberately avoiding N+1 client fan-out. The 30-second polling is what keeps the landing numbers fresh without the WebSocket dependency.

07 · Pages — Modules, Test Cases list, Test Case detail

> Files: `views/modules.tsx` (511 lines), `views/test-cases.tsx` (411), `views/test-case-detail.tsx` (626). These three screens cover the authoring core of TCMS: organising features, writing cases, and evolving them version by version.

---

1. Modules view (`'modules'`) — the feature tree

Purpose & layout

Two panels. **Left:** the module tree (roots → submodules, expand/collapse, per-node test-case-count badges). **Right:** the selected module's detail — metadata, owner, submodule chips, its test cases, and module-level actions.

Data

Query	Endpoint	Notes
`['modules','tree']`	`GET /api/modules`	`{ tree, flat }` — the left panel + every module picker in the app
`['modules','detail',selectedId]`	`GET /api/modules?id=…`	`{ module, testCases, submodules }`, `enabled: !!selectedId`
`['users','all']`	`GET /api/users?pageSize=200`	for the reviewer picker in the submit dialog

State

`selectedId`, `expanded: Set<string>` (tree open/closed), `createOpen` + `createParent: string|null`, `deleteId`, `submitReviewModule: {id,name}|null`.

Derived permissions (inline role logic)

- `canManage = admin || team_lead` → shows **"Submodule"** create button (and root create).
- `canSubmit = role !== 'viewer'` → shows **"Submit for Review"**.
- `canDeleteSelected = isCreator || isSenior` → the delete button is **always visible but disabled** otherwise, with tooltip "Only the creator or a senior manager can delete this module" plus an inline amber warning (`ShieldAlert`).

Interactions, one by one

Action	What happens
Click a tree row	selects module → right panel fetches detail
Click a submodule chip	selects it and auto-expands the parent branch
Create Submodule / Root dialog	fields Name (required — empty → `toast.error('Name is required')`), Description; info box "Creator / Owner: {you}" explaining the delete rule. `POST /api/modules` **without** `ownerId` (server forces creator=owner). Success: `Module created — you are the owner`; invalidate `['modules']`
Submit for Review dialog	Reviewer select (team_leads/admins only; placeholder "Auto-assign (first available team lead)"), optional due date, checkbox **"Include submodules" (default true**; description notes approved/active cases are skipped). `PUT /api/modules?id=…&action=submit-review`. Success toast `Submitted {N} test case(s) for review`; invalidates `['modules','test-cases','reviews','module-reviews']`; **auto-navigates** to `module-review-detail { id }`
Delete confirm (`AlertDialog`)	warns about cascade to submodules and orphaned cases. `DELETE /api/modules?id=…`; invalidate `['modules']`; clear selection if deleted
Case row click / "View all →"	`navigate('test-case-detail', { id })` / `navigate('test-cases')`

Things to notice

The tree is rendered recursively from `tree` — materialised `path`/`depth` fields make indention trivial. The create dialog deliberately never asks who owns the module; ownership follows creation, which is what makes the delete-permission rule coherent.

---

2. Test Cases list (`'test-cases'`) — filter, find, export

Layout

Toolbar (search + 3 filter selects + Export CSV + New Test Case) → table → shared `<Pagination>` footer.

Data

- `['test-cases','list',{ search, statusFilter, priorityFilter, moduleFilter, page }]` → `GET /api/test-cases?search&status&priority&moduleId&page&pageSize=20` — filters are **part of the query key**, so every combination caches independently.
- `['modules','tree']` for the module filter + create dialog.

State & behaviours

Control	Detail
Search input	placeholder "Search by title..."; typing resets `page` to 1 (server `contains` search — case-sensitive)
Status select	All + the 6 statuses
Priority select	All + P0–P3
Module select	All + flat module list
Export CSV	client-side blob from **current page rows**; headers `ID, Title, Module, Priority, Status, Tags, Author, Updated`; file `test-cases-YYYY-MM-DD.csv`; toast `Exported CSV` / `No data to export`
New Test Case	**hidden for viewers**; opens the create dialog

Table columns

ID (last 6 chars), Title (truncated 60), Module, Priority badge, Status badge, Tags (first 2 + `+N`), Author, Updated. Row click and eye icon → `navigate('test-case-detail', { id })`. Empty state: "No test cases match your filters".

CreateTestCaseDialog

Fields: Title (required), Module select (optional), Priority (default `P2`), Tags (comma-separated → array), Description, Preconditions, **Steps editor** (numbered rows of Action + Expected result; "Add step"; remove disabled at 1 step; empty rows filtered out), Expected Result, author info box. Submits `POST /api/test-cases` **without** `authorId` (server: author = me, status = draft, version = 1). Success: toast `Test case created — you are the author`, invalidate `['test-cases']`, close, **navigate to the new case's detail page**.

---

3. Test Case detail (`'test-case-detail'`) — read, edit, version, review

Opened with `params.id`. The richest screen in the app: five queries, an inline editor, a submit-for-review dialog, soft delete, and three side panels.

Data (5 queries)

Key	Endpoint
`['test-cases','detail',id]`	`GET /api/test-cases?id=`
`['test-cases','history',id]`	`…&action=history`
`['test-cases','reviews',id]`	`…&action=reviews`
`['activity','test_case',id]`	`GET /api/activity-logs?action=entity&type=test_case&id=`
`['modules','tree']` + `['users','all']`	pickers

Derived permissions

```
isAuthor   = tc.authorId === user.id
isSenior   = admin || team_lead
canEdit    = isSenior || (isAuthor && role === 'qa_engineer')
canDelete  = (isAuthor || isSenior) && status !== 'deprecated'
canSubmit  = role !== 'viewer' && (status === 'draft' || status === 'needs_revision')
```

Main column (read mode)

Badges: priority, status, `v{version}` · module name · tags · then sections Description → Preconditions → **Steps table** (# / Action / Expected Result) → Expected Result. While under review the header additionally shows Awaiting review (`in_review`) or Approved (`approved`) chips.

Buttons & flows

Button (when visible)	Flow
Edit (`canEdit`)	swaps the main column to the inline `TestCaseEditor` — Title, Module, Priority, **Reviewer (assignee)** select (team_lead/admin only, placeholder Unassigned), Tags, Description, Preconditions, Steps editor, Expected Result. Save → `PUT /api/test-cases?id=` → server snapshots old state into history and bumps `version`. Client invalidates detail + history + list, toasts `Test case updated`, exits edit mode
Delete (`canDelete`)	`DELETE` → soft delete → toast `Test case deprecated` → navigate back to the list
Submit for Review (`canSubmit`)	dialog: Reviewer (pre-filled with `tc.assigneeId`, placeholder Auto-assign) + optional due date → `PUT …&action=submit-review` → toast `Submitted for review`, invalidates detail/reviews/queue
Back to Test Cases	navigate

Side panels (right column)

1. **People** — Author (+created date), Reviewer/assignee, Last edited by.
2. **Version History** — each entry `v{n}`, who changed, summary (`"Updated: …"` / `"Field update"`), `timeAgo`; scrollable. Data = TestCaseHistory snapshots (they hold the previous state of each version).
3. **Review Cycles** — per ReviewRequest: `Cycle {n}` badge, review status badge, reviewer, and the nested ReviewEvent timeline with verdict badges + comments + flagged-field chips.
4. **Activity** — first 8 entity-scoped activity logs with `actionColor` badges.

The versioning mental model (end to end)

Edit v3 → server writes history row `version: 3` containing the v3 snapshot → case becomes v4. The history card shows where each version came from; the snapshots make "what did it look like before" answerable forever (rows are only deleted with the case itself, which never happens — delete is soft).

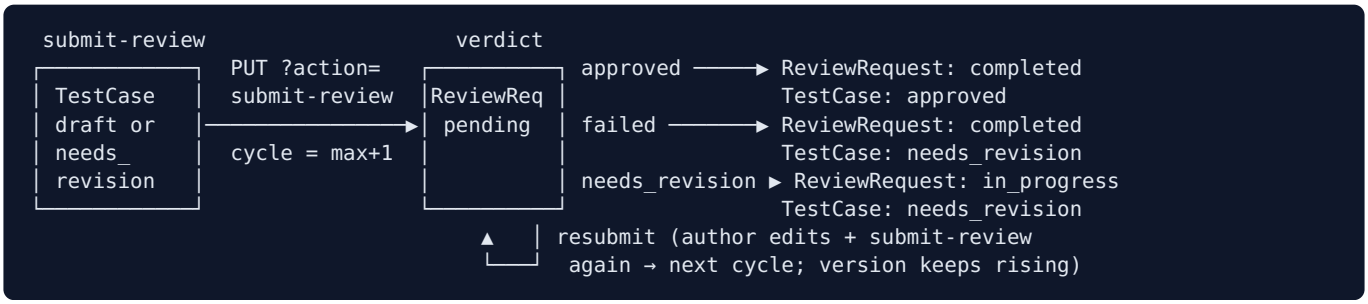
08 · Pages — the Review workflow (queue → detail → verdicts)

> Files: `views/reviews.tsx` (253), `views/review-detail.tsx` (347), `views/module-review-detail.tsx` (415). Together they implement the quality gate of the whole system. Read the state machine first — the three screens are just its UI.

---

1. The review state machine (server-enforced)

Single test-case review



Key invariants:

- Submitting is only possible from `draft` / `needs_revision` (UI) and is only created by the API from those statuses' cases (eligible set).
- **Verdicts may only be recorded by the assigned reviewer (or an admin)** — `perform_review` permission + ownership check.
- `needs_revision` keeps the ReviewRequest open (`in_progress`) so the same request carries the rework conversation; the **cycle** counter increments per resubmission.
- A verdict always notifies the **author**; comments are mandatory unless `approved`; flagged fields are structured labels chosen from a fixed list.

Module (bulk) review

```
submit-review on module ─▶ ModuleReviewRequest(pending, cycle N)
                            └─▶ ReviewRequest per eligible case (same cycle,
                                moduleReviewId set) — all cases → in_review
first child verdict ──────▶ parent → in_progress
ALL children completed ──▶ parent → completed (+ notify requester)
"Approve All" ────────────▶ every pending child → approved event,
                            case → approved, parent → completed (one shot)
```

---

2. Reviews queue (`'reviews'`) — the reviewer's home

Tabs

Tab	Data	Who sees it
Module Reviews (default)	`['module-reviews','queue']` — mine, open	everyone
Single Test Cases	`['reviews','queue']` **filtered to** `!r.moduleReviewId` (children of module reviews are handled on their parent's page)	everyone
All Reviews	`['reviews','all']` (pageSize 100) + `['module-reviews','all']`, `enabled: tab==='all'`	**admin/team_lead only** — the tab isn't rendered otherwise; the API also enforces 403

Both queue queries poll every 30 s and are the same keys the **sidebar badge** uses — one fetch feeds two UIs (badge count = standalone queue length + module queue length).

Cards

- **ModuleReviewCard** (green left border): module name, `Module review · Cycle N`, status badge, case-count badge, requester, due date with **overdue detection** (red
  - `AlertCircle` when `due < now && status !== 'completed'`).
- **ReviewCard**: case title + module, status/priority badges, `Cycle N`, requester, same due/overdue logic.
- Click → `navigate('module-review-detail' | 'review-detail', { id })`.

Empty states: "No pending module reviews / You're all caught up!", "No individual test-case reviews pending". Loading: `StateLoadingCards`.

---

3. Review detail (`'review-detail'`) — judge one case

Opened with `params.id`. One query: `['reviews','detail',id]` → `GET /api/reviews?id=` which embeds the full serialized test case + events.

Who can act

```
isReviewer = review.reviewerId === user.id || user.role === 'admin'
canAct     = isReviewer && review.status !== 'completed'
```

Non-reviewers (and everyone after completion) see the same page read-only — content plus history, but no verdict buttons.

Layout

Header: case title, Cycle / review-status / priority badges, submitted-by and reviewer avatars, due date. Body: read-only Description / Preconditions / **Steps table** / Expected Result. Below: **Review History** — every ReviewEvent with verdict-colored left border, reviewer, `timeAgo`, comment, and `flaggedFields` chips (guarded by `Array.isArray`). Buttons: **"View Test Case"** → `test-case-detail {id}`, **"Back to Reviews"**.

VerdictDialog

Opened by three buttons: **Approve** (green) / **Fail** (red) / **Needs Revision** (outline). Dialog title adapts (`Approve Test Case` / `Fail Test Case` / `Request Revision`).

Field	Rules
Flagged Issues checkbox grid	rendered **only when verdict ≠ approved**; fixed options: `Steps unclear, Wrong expected result, Missing preconditions, Missing tags, Incomplete description, Ambiguous title`
Comment textarea	**required unless approved** (client blocks submit + toast)

Submit → `reviewsApi.verdict(id, { verdict, comment, flaggedFields })` → `POST /api/reviews?id=` → server does the full state-machine step (event + statuses + module aggregation + notifications, see doc 04 §7). Client toasts `Verdict submitted: needs revision`, invalidates `['reviews']` + `['test-cases']`, navigates back to the queue.

---

4. Module review detail (`'module-review-detail'`) — the bulk workspace

Opened with `params.id`. One query: `['module-reviews','detail',id]` — the payload embeds every child case review **with its full test case** (this is the one screen where a reviewer can do everything without leaving the page).

Header stats (derived client-side)

`Total Cases` (slate) · `Approved` (emerald — children with an `approved` event) · `Needs Attention` (amber — `failed` / `needs_revision`), plus pending count.

Actions

Control	Behaviour
Approve All ({pendingCount})	visible only to `canAct` users when pending > 0. Dialog explains consequences ("each test case will move to 'approved' status and the module review will be marked completed"), optional comment (placeholder Bulk approved with module review.). `POST /api/module-reviews?id=…&action=approve-all` → toast `Approved {N} test case(s)`; invalidates detail + `['test-cases']` + `['reviews']`
Case row chevron	expands an inline panel: read-only Description / Preconditions / **Steps as ordered list** (action → expected) / Expected Result + per-case review history
Per-case **Approve**	immediate `reviewsApi.verdict(cr.id, { verdict: 'approved' })` — no dialog needed; toast `Test case approved`; invalidate detail + test-cases. (Server aggregation may complete the parent — surfaced on refetch)
Per-case **Fail** / **Needs Revision**	`window.prompt` for the comment (required; toast error if empty) → verdict `failed` / `needs_revision`. Quirk: prompt-based, unlike the rich dialog on review-detail.

Permissions mirror review-detail exactly (`reviewerId === me || admin`, and the parent must not be `completed`). After the last pending child is resolved anywhere, the parent flips to `completed` server-side; the header then renders fully read-only.

Why the two-level design exists

A module review is one reviewer, one due date, one conversation for up to dozens of cases: the umbrella row carries the deadline/cycle, the children carry the per-case verdicts, and aggregation (automatic completion +

requester notification) removes the need to manually close anything. The All-Reviews tab (doc 08 §2) lets leads audit both levels afterwards.

09 · Pages — Bugs (Kanban), Tasks (daily log), Reports

> Files: `views/bugs.tsx` (492), `views/tasks.tsx` (546), `views/reports.tsx` (418). Three working surfaces: triage by drag & drop, personal accountability per day, and management analytics.

---

1. Bugs view (`'bugs'`) — board + table

Layout & state

Header tabs **Board / Table** (`view: 'kanban' | 'table'`, default board) and **"Report Bug"** (hidden for viewers). Data: `['bugs','list']` → `GET /api/bugs?pageSize=200` (one fetch powers both views — the dataset is small by design).

Kanban board (drag & drop with @dnd-kit)

- Five columns, keyed by `BUG_STATUSES = open, in_progress, resolved, closed, wont_fix`; each is a `useDroppable` with a count badge and a "Drop here" placeholder; columns highlight green when a card hovers (`isOver`).
- `DndContext` + `PointerSensor` (activation distance 5px — distinguishes click vs drag).
- **`handleDragEnd`**: if the status changed → `bugsApi.update(bugId, { status })` → toast `Moved to "{status}"`; invalidate `['bugs']`; failures toast via `handleApiError(err, 'Failed to move')`. **This is the only "drag = mutation" in the app** — and note it means any authenticated user can move any bug (deliberate; the API gates only assignment and deletion).

Table view

Columns: ID (last 6), Title, Severity badge, Status badge, Assigned, Module, Reported (`timeAgo`). Row click opens the detail sheet.

CreateBugDialog ("Report a Bug")

Fields: Title (required), Severity (default `medium`), Environment (default `staging`), Module select, **Assign To** (users filtered to team_lead/admin/ qa_engineer), Description, Steps to Reproduce, Expected Behaviour, Actual Behaviour. `POST /api/bugs` → toast `Bug reported`, invalidate `['bugs']`. If an assignee was chosen the server notifies them (`bug_assigned`).

BugDetailSheet (right-side Sheet, max-w-lg)

Shows everything: badges, `#{id6}`, reporter + timestamp, Description, Environment, Steps, Expected / Actual, module + assignee grid.

- **Edit** toggles inline selects (Status / Severity / Assignee) + Save (`bugsApi.update` → `Bug updated`). Moving status to resolved/closed makes the **server** stamp `resolvedAt`.
- **Delete** (red) → `DELETE /api/bugs?id=` — **admin only** server-side; no confirmation dialog (known quirk), toasts `Bug deleted`.

---

2. Tasks view (`'tasks'`) — one report per user per day

The model

A TaskReport = a date + an array of task lines (`description`, `minutes`, optional link to module/test-case/bug, `status: done|in_progress|blocked`) + notes. Unique per (user, date) — the screen is an **editor for exactly one day of one person**.

Who can edit whom

```
isManager = admin || team_lead
managers  → user switcher; viewing someone else = READ-ONLY
            (banner: "You are viewing another user's report. Switch back to yourself to make edits.")
everyone  → full editing of their own report for any date
```

(The API mirrors this: upsert always owns the row; PUT is owner-only.)

Data

- Self (non-manager): `['task-reports','my']` → `GET ?action=my`.
- Manager: `['users','list','all']` + `['task-reports','user',effectiveUserId,reportDate]` → `GET ?userId&dateFrom=date&dateTo=date`.
- Reference data: modules tree, light test-case list (pageSize 200), light bug list — for the link pickers.

The form

- Header: date input (`reportDate`, timezone-safe `todayISO()`) + manager-only user select (`fullName · email`).
- Task lines: Description · Minutes · **Link type** (`No link / Module / Test case / Bug` — switching resets all three ids) · Status · remove. A linked line renders a second picker row ("Linked module for entry #N", modules indented by depth).
- **"Add task"** appends a blank line (`status: in_progress`, 0 min).
- Notes card: free text.
- The whole form component is **re-keyed** by `` `${effectiveUserId}-${reportDate}-${existingReportId ?? 'new'}` `` — when the server data for a day changes, React remounts the form and local state resets cleanly (a deliberate alternative to effect-syncing).

Save & export

- **Save** → `taskReportsApi.upsert({ reportDate, tasks, notes })` → `POST` (find-then- update upsert by user+date; server recomputes `totalMinutes`) → toast `Task report saved`; invalidate `['task-reports']`. Hidden when read-only.
- **Export CSV** (disabled when empty): rows `Date, Description, Minutes, Status, Linked Type, Linked ID` + notes + total; file `tasks-{reportDate}.csv`.
- Sticky footer: `Total time: {minutesToHuman(totalMinutes)}` + task count.

---

3. Reports view (`'reports'`) — team analytics

Access

`admin` / `team_lead` only — the sidebar hides the item and the view renders a destructive "Access restricted" alert for others (API: `view_all_task_reports`).

Data

One query: `['stats','reports']` → `GET /api/stats?action=reports` → `{ testCasesByStatus, testCasesByPriority, bugsByStatus, weeklyHours }` (weekly = last 30 days, Sunday-based weeks).

Tabs

Team summary — pivot of `weeklyHours` into a per-user × per-week table (totals column, `–` for zero) + vertical bar chart of total hours per user (emerald `#10b981`, names truncated at 18 chars).

Test case metrics — three recharts: status donut (slate/amber/orange palette), priority donut (`P0 #ef4444, P1 #f97316, P2 #eab308, P3 #0ea5e9`), bugs-by-status bar (`open #f43f5e, in_progress #f59e0b, resolved #10b981, closed #64748b, wont_fix #a1a1aa`) with legends.

Known limitation (documented honestly)

The two date inputs in the Weekly hours card header (`dateFrom`, `dateTo`) are **cosmetic** — `statsApi.reports()` takes no parameters, so changing them refetches nothing. The API's `?action=summary` on task-reports does accept ranges; wiring the UI to it is a straightforward future improvement (file under "next steps", doc 13).

Export

"Export PDF" → `window.print()` with a `.print-area` class on the metrics grid — the browser's print dialog is the PDF pipeline (zero dependencies).

10 · Pages — Activity Log, Notifications, Users (admin), Profile

> Files: `views/activity-log.tsx` (463), `views/notifications.tsx` (253), `views/settings-users.tsx` (522), `views/settings-profile.tsx` (322). The governance corner of the app: the audit trail, the inbox, account administration, and self-service settings.

---

1. Activity Log (`'activity-log'`) — the audit trail UI

Access

`admin` / `team_lead` (sidebar-hidden for others; API enforces `view_activity_logs`).

Data

`['activity-logs',params]` → `GET /api/activity-logs?page&pageSize=50&userId&entityType&action&dateFrom&dateTo`. Plus `['users','list','activity']` for the user filter.

Filters card (each change resets page to 1)

From / To date pickers · User select (All users) · Entity select (`module, test_case, review, bug, task_report, user`) · Action select (`CREATE, UPDATE, DELETE, STATUS, LOGIN, REVIEW`) · **Reset** button.

Table

Column	Detail
Timestamp	`formatDateTime`
User	avatar + name, or **System** when `userId` is null
Action	colored badge via `actionColor` (prefix-based: CREATE emerald, UPDATE amber, DELETE rose, STATUS sky, LOGIN slate, REVIEW purple)
Entity	badge with `entityType`
Name	**clickable** (row-independent, `stopPropagation`) → cross-navigation: `test_case` → case detail `{id}`, `bug` → bugs view, `module` → modules, `review` → reviews
Changes	`{n} field(s)` badge when a `diff` exists, else `—`

Row click → **ActivityDetailDialog**: title = action + entity badges + entity name; description = actor, timestamp, optional IP; body = Changed fields (N) with a **DiffRow per field** — red "Before" panel (strikethrough) vs green "After" panel, values JSON-pretty-printed when not plain strings. Empty diff → "No field-level changes recorded."

Pagination: custom Prev/Next disabled while `isFetching`, caption `{total} entries · page {page} of {totalPages}` (deliberately not the shared `<Pagination>` component — it needs the fetching-disable behaviour).

Where the data comes from (remind)

Every route's `logActivity()` call wrote `beforeSnapshot`/`afterSnapshot` and a computed `diff` (`{field: {before, after}}`, shallow JSON comparison). Nothing writes to this table from the UI — it is append-only by design.

---

2. Notifications view (`'notifications'`) — the inbox

Data source (special!)

This view does **not** run its own list query. The **Zustand notifications store is the single source of truth**: seeded once by `useNotificationSocket` (fetch of latest 50), then updated by WebSocket pushes and local mark-read/remove actions. The API's paginated `GET /api/notifications` remains the canonical store; the UI list is a cache.

Screen

- Header: "Notifications" + subtitle `You have N unread notification(s).` or "You're all caught up." + **Mark all read** (only when unread > 0; POSTs `?action=read-all`, updates store, invalidates `['notifications']`, toasts).
- List items: type-specific icon + color — review_* → ClipboardCheck, `bug_assigned` → Bug, `test_case_updated` → FileEdit, `module_created` → FolderTree, `comment_added` → MessageSquare, fallback Bell. Unread items get a tinted background + emerald dot.
- **Item click** (or Enter/Space — keyboard accessible): mark read via PATCH (errors silently ignored) → **deep-link by `entityType`**: `test_case` → case detail `{entityId}`, `bug` → bugs, `module` → modules, `review` → reviews, else no-op.
- Hover **X** button → `DELETE /api/notifications?id=` → store remove + invalidate.
- Empty state: green CheckCheck icon, "You're all caught up", caption "New notifications will appear here in real time."

The same items (first 10) appear in the **topbar bell popover**, sharing the store — marking read in either place updates both instantly.

---

3. Users (`'settings-users'`) — admin console

Access

`admin` only: sidebar-gated, view renders "Admins only" alert otherwise, and every `/api/users` method returns 403 for non-admins regardless of the UI.

Data & mutations

`['users','list',params]` → `GET /api/users?page&pageSize=50&role&isActive`; create/update/delete mutations → POST / PUT / DELETE with `{ role? }`, `{ isActive? }` etc.; all invalidate `['users']`; toasts `User invited` / `User updated` / `User deactivated`.

Filters & table

Role select (All/Admin/Team Lead/QA Engineer/Viewer) and Status select (All/Active/ Inactive), both reset page. Table: avatar + name (**"(you)"** marker) + email · **inline Role select** · **Active Switch** · Member since · trash button.

Self-protection rules (client + server agree)

You cannot change your **own** role, deactivate yourself, or delete yourself — the controls are disabled for the current user's row (`Cannot delete yourself` server-side).

Invite dialog

"Invite a new user" — Email, Full name, **Temporary password** (min 8, client-blocked otherwise), Role select (admin option **is** available here — admin-created users can be any role). They can sign in immediately.

Delete dialog

"Delete user?" — explains the soft-delete semantics verbatim: "This will deactivate {name} and revoke their access. Their historical data (test cases, bugs, reports) will be preserved." Confirm is red; row kept with `isActive: false` (login rejected).

---

4. Profile (`'settings-profile'`) — self service

Account card

Avatar initials, full name, email, role badge (`roleColor`), "Member since" date. Read-only identity (email change is deliberately impossible: the API whitelist never includes it; the input is disabled with a caption "Email cannot be changed. Contact an admin if it needs updating.").

ProfileForm

Fields: Full name (required), Email (disabled), Avatar URL (`type=url`), Telegram chat ID, three **Switch** toggles — Email notifications, Telegram notifications, In-app notifications (only `notifyInapp` has a live server-side effect today: `createNotification` skips rows for users with it off; the other two are stored preferences). Save → `PUT /api/auth` (whitelist: `fullName, avatarUrl, telegramChatId, notifyEmail, notifyTelegram, notifyInapp`) → `authStore.setUser(updated)` so the topbar/name update instantly → toast `Profile updated`.

PasswordForm

Current / New / Confirm password. Client checks: new ≥ 8 chars, confirm matches (mismatch shows a persistent destructive alert while both fields are non-empty and differ). Submit → `PUT /api/auth?path=password` → server re-verifies the current password (400 `Current password is incorrect` on mismatch), re-bcrypts, logs `UPDATE/user`. Toast `Password changed`, fields cleared. Other sessions keep working until their access token expires (15 min) — a reasonable trade-off documented here.

---

5. How these four views interlock (the governance loop)

```
every mutation ──▶ ActivityLog (audit)  ──▶ Activity Log view (leads audit it)
      │
      └─▶ Notification row (inbox) ──▶ bell popover / Notifications view
                  │                          (click = deep link to the entity)
                  └─▶ WS push (instant) ── otherwise visible on next fetch
Users view controls who can log in and with which role ──▶ every gate in docs 03/06–09
Profile keeps identity + delivery preferences truthful ──▶ notification skip logic
```

11 · End-to-End Flows — click to database and back

> The complete stories behind the six things this system does. Each flow names every file, function, endpoint, table and status change in order. These are the "movie versions" of docs 04–10; use them to trace or debug any real interaction.

Flow 1 · From page load to logged-in dashboard

```
1. GET /                      → Next.js serves layout.tsx + page.tsx (static shell)
2. <Providers>                → QueryClient (staleTime 30s, retry 1, no refetch on focus)
3. <AuthGate>                 → useAuthStore.initialize()
     localStorage: tcms-user + tcms-access exist?
     ├─ no  → booted; AppShell renders <AuthScreen>              [auth-screen.tsx]
     └─ yes → GET /api/auth (Bearer token + cookie)
                ├─ 200 → publicUser → setUser → AppShell
                └─ 401 → tryRefresh() (single-flight)
                           ├─ ok → new pair saved, request retried → 200
                           └─ fail → clearSession() → reload → <AuthScreen>
4. AppShell                   [app-shell.tsx]
     - useNotificationSocket(user.id)  → WS connect + seed notifications (50)
     - <Sidebar> prefetch ['reviews','queue'] + ['module-reviews','queue'] (30s poll)
     - CurrentView = 'dashboard'
5. Dashboard                  [dashboard.tsx]
     GET /api/stats?action=dashboard → 6 parallel Prisma queries → KPIs + charts
6. User clicks demo fill + "Sign in" (first visit):
     POST /api/auth?path=login → bcrypt check → createSession
       → RefreshToken row (7d) → logActivity(LOGIN/user)
     → setSession → cookie + localStorage → toast "Welcome back!"
```

Tables touched: `RefreshToken` (insert), `ActivityLog` (LOGIN). **What can fail:** expired refresh → clean login screen; deactivated user → 401 at login.

Flow 2 · Create a test case → edit it (versioning) → see its history

```
1. Test Cases view → "New Test Case"          [test-cases.tsx]
     dialog validates Title → POST /api/test-cases
       server: can('create_test_case') → status 'draft', authorId=me, version=1
       → logActivity(CREATE/test_case, after {title, moduleId, priority})
2. Client: toast, invalidate ['test-cases'], navigate('test-case-detail',{id})
3. Detail page → "Edit"                       [test-case-detail.tsx]
     canEdit = isSenior || (isAuthor && qa_engineer)
     editor (inline) → PUT /api/test-cases?id=
       server route order:
         a. 404 check → b. permission check
         c. TestCaseHistory INSERT {snapshot of v1 fields, version: 1,
            changeSummary: "Updated: <new title>" | "Field update"}
         d. UPDATE ... version: 2, lastEditedById=me
         e. logActivity(UPDATE/test_case, before=full v1, after=full v2) → diff computed
         f. assigneeId changed? → notification 'test_case_updated' to assignee
4. Client invalidates detail + history + list → the page now shows v2
5. Side panel "Version History" lists v1 (the snapshot row) and the current version
```

Tables: `TestCase` (insert, update), `TestCaseHistory` (insert), `ActivityLog`, `Notification` (optional). **Invariant:** an edit NEVER changes `status`; only reviews do.

Flow 3 · Submit for review → verdict → author notified (single case)

```
1. Author (qa1) on case (status draft) → "Submit for Review"
     dialog: reviewer (prefilled = case.assigneeId) + optional due date
     PUT /api/test-cases?id=…&action=submit-review
       server: can('submit_review') → resolve reviewer (body → assigneeId →
         first active lead/admin) → cycle = max+1 (say 1)
       → ReviewRequest INSERT (pending, cycle 1)
       → TestCase UPDATE status='in_review' (version unchanged!)
       → logActivity(REVIEW_SUBMITTED/test_case)
       → createNotification(reviewer, 'review_assigned') → Notification row
              → fire-and-forget POST localhost:3003/notify → WS room user:{reviewerId}
2. Reviewer's browser (WS connected): 'notification' event → bell badge +1 instantly
   (WS down? row is in the DB; 30s queue poll or next fetch shows it)
3. Reviewer → Reviews view → "Single Test Cases" tab → card (Cycle 1, due date)
     → navigate('review-detail',{id})               [review-detail.tsx]
     GET /api/reviews?id= → review + case + events
4. Reviewer clicks "Needs Revision" → dialog:
     flagged fields: [Steps unclear], comment required (client + server enforce)
     POST /api/reviews?id= body {verdict:'needs_revision', comment, flaggedFields}
       server: can('perform_review') ∧ (assigned ∨ admin)
       → ReviewEvent INSERT
       → ReviewRequest: pending → in_progress   (stays open for the rework loop)
       → TestCase: in_review → needs_revision
       → logActivity(REVIEW_VERDICT/test_case, before {status: in_review})
       → createNotification(AUTHOR, 'review_needs_revision', body incl. comment)
5. Author: bell +1 → click → navigates (entityType 'review' → Reviews view);
   case page shows status needs_revision, cycle 1 with one event in Review Cycles
6. Author edits (Flow 2; version++) and resubmits → ReviewRequest cycle 2 (pending)
7. Reviewer approves → ReviewRequest completed; TestCase approved;
   author gets 'review_approved'
```

Tables: `ReviewRequest` ×2, `ReviewEvent`, `TestCase` (status only), `ActivityLog`, `Notification`. **Response shape the client receives:** `{ event, reviewStatus, testCaseStatus }`.

Flow 4 · Module-level bulk review (and auto-completion)

```
1. Team lead opens Modules → picks "Authentication" → "Submit for Review"
     dialog: reviewer=auto (first active lead), due date, includeSubmodules ✓
     PUT /api/modules?id=…&action=submit-review
       server: eligible = cases of module + descendants with status
         draft|needs_revision (say 4 cases)
       → cycle = max(module cycles)+1 = 1
       → ModuleReviewRequest INSERT (pending, cycle 1)
       → ReviewRequest INSERT ×4 (pending, moduleReviewId=parent, cycle 1)
       → TestCase UPDATE ×4 → in_review
       → logActivity(REVIEW_SUBMITTED/module); notify reviewer (module_review)
     client: toast "Submitted 4 test case(s) for review"
            → navigate('module-review-detail',{id})
2. Reviewer opens the workspace           [module-review-detail.tsx]
     header stats: Total 4 · Approved 0 · Needs Attention 0
     expands case rows → per-case Approve (immediate) or Fail/Needs-Revision (prompt)
     each verdict = Flow 3 step 4, PLUS module aggregation check:
       after each verdict the server asks "are ALL sibling reviews completed?"
         ├─ no  → parent ModuleReviewRequest → in_progress
         └─ yes → parent → completed
                → notify REQUESTER 'review_approved' ("Module review completed")
3. Shortcut: "Approve All (4)" → POST ?action=approve-all {comment?}
       → per pending child: ReviewEvent(approved) + review completed + case approved
       → parent → completed → notify requester "Module review approved"
```

```
       → response {approvedCount: 4, moduleReviewStatus: 'completed'}
 4. Everything refetches; the parent page is now fully read-only
```

Tables: `ModuleReviewRequest`, `ReviewRequest` ×N, `ReviewEvent` ×N, `TestCase` ×N, `ActivityLog`, `Notification`.
Invariant: children inherit the parent's cycle; the umbrella auto-completes — nobody closes it manually.

Flow 5 · Bug reported → triaged by drag & drop → resolved

```
 1. Bugs view → "Report Bug"                    [bugs.tsx]
      dialog: title, severity, environment, module, assignee, descriptions
      POST /api/bugs → server: can('create_bug') → reportedById=me
        → optional BugTestCaseLink rows (none from this dialog today)
        → logActivity(CREATE/bug)
        → assigned? → notification 'bug_assigned' to assignee
 2. Kanban board: assignee drags card open → in_progress
      DndContext onDragEnd → status changed?
      PUT /api/bugs?id= body {status:'in_progress'}    (any authenticated user may)
        → logActivity(UPDATE/bug, before/after full row → field diff)
 3. Later: drag to resolved → PUT {status:'resolved'}
      server stamps resolvedAt = now() (client value ignored)
 4. Admin cleanup of a spam bug: sheet → Delete → DELETE (admin only, hard delete)
```

Tables: `Bug` (insert, update, delete), `BugTestCaseLink` (optional), `ActivityLog`, `Notification` (optional). **Quirk to remember:** delete has no confirmation dialog; assignment requires `assign_bug`, movement does not.

Flow 6 · Daily task report → team analytics

```
 1. Tasks view (qa1, date = today)              [tasks.tsx]
      form keyed by {userId}-{date}: 2 task lines
        line 1: "Wrote cases for login flow", 120m, linked Module "Authentication - Login", done
        line 2: "Investigated Safari bug", 60m, linked Bug #{…}, in_progress
      notes: "Need to sync with frontend."
      Save → POST /api/task-reports  (upsert by (me, reportDate))
        server: can('create_task_report') → findFirst(user,date)
          ├─ none → CREATE (first save today)
          └─ found → UPDATE tasks/notes, totalMinutes = Σ = 180
        → logActivity(CREATE|UPDATE/task_report, after/before {totalMinutes})
 2. Same screen later: refetch shows saved lines; edits upsert again (no duplicates,
    DB unique constraint [reportedById, reportDate] is the backstop)
 3. Manager opens Reports → Team summary
      GET /api/stats?action=reports (view_all_task_reports)
        weeklyHours = last 30 days grouped by user + Sunday-week → 3h for qa1 this week
      QA1 row appears in the pivot table + hours bar chart
 4. CSV: qa1 clicks Export on Tasks → tasks-2026-09-04.csv (client-side blob)
```

Tables: `TaskReport` (upsert), `ActivityLog`. **Guard rails:** non-managers only ever see their own rows (API force-filters); PUT is owner-only (admins included).

Flow 7 · A notification's life (realtime vs fallback)

```
server createNotification(user, type, …)     [lib/notifications.ts]
  1. user exists? active? notifyInapp on?  (else: silent skip)
  2. Notification INSERT (isRead=false)
  3. fetch POST localhost:3003/notify {userId, …payload}   ← not awaited
       WS service: io.to('user:'+userId).emit('notification', payload)
         client hook [use-notification-socket.ts]: store.addNotification
           → topbar bell badge increments immediately
```

```
      (Vercel: fetch fails → console.warn ONLY. DB row remains; user sees it via
       the 30s queue polls or on next navigation/refresh. Zero functional loss.)
  4. user clicks it (bell popover or Notifications view)
       → PATCH /api/notifications?id= (isRead=true) + store.markRead
       → navigate by entityType (test_case/bug/module/review)
  5. "Mark all read" → POST ?action=read-all (updateMany)
```

Tables: `Notification`. **Failure modes:** WS off → delayed visibility only; notifyInapp off → nothing at all (by user choice); every step is try/catch-guarded so notifications can never break a business operation.

Flow 8 · Admin manages a user (invite → promote → offboard)

```
invite:  Users view → "Invite a new user" → POST /api/users (admin only)
           duplicate email → 400; zod: password ≥ 8; role required (any role)
           → logActivity(CREATE/user)
promote: inline Role select → PUT /api/users?id= {role:'team_lead'}
           → effect: next request that user makes carries the new role in its JWT…
           NOTE: the old access token (≤15 min) still has the old role until expiry
           or refresh — document this when testing permissions!
offboard: trash → confirm → DELETE → isActive:false (soft)
           → login now returns 401 Invalid credentials
           → history preserved: ActivityLog.user is SetNull-safe, author names remain
             via stored entityName / relation snapshots
self:     role/active/delete controls disabled for your own row ("Cannot delete yourself")
```

Cross-flow invariants (the "physics" of TCMS)

1. **Only reviews change case status** (and soft-delete). Edits bump version, never status.
2. **Version up = history row written first.** You can always answer "what changed?".
3. **Every mutation → ActivityLog; every human-relevant mutation → Notification.**
4. **The UI hides forbidden actions; the API enforces them.** Bypassing the UI never bypasses a permission.
5. **No optimistic writes.** Server confirms → toast → cache invalidation → UI updates.
6. **Audit and notifications never break the operation** (both fully try/catch-wrapped).
7. **Deletes are deliberate per entity:** module hard (cascade), case soft (deprecated), bug hard (admin), user soft (deactivated).

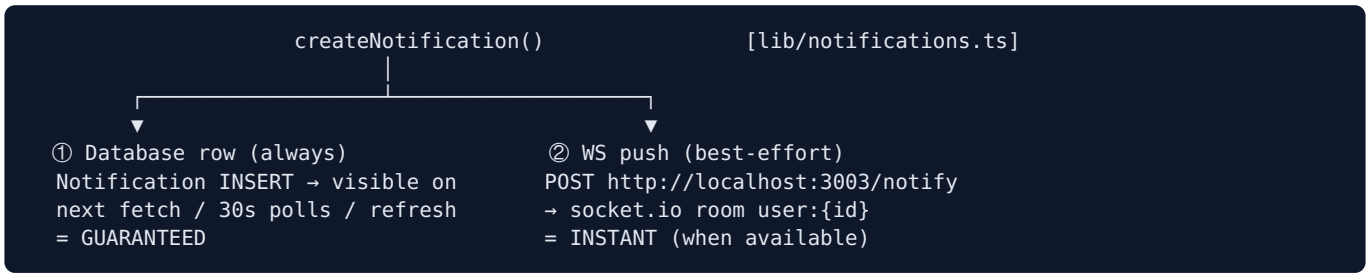
12 · Realtime Service & Deployment Architecture

> How the optional WebSocket mini-service works, and how the whole system maps onto Vercel + Neon (the deployment documented step-by-step in `DEPLOYMENT-GUIDE.md`).

---

1. The realtime problem & the graceful-degradation answer

TCMS wants "bell badge increments the instant someone assigns you a review". Serverless platforms (Vercel functions) cannot hold persistent WebSocket connections, so the project splits delivery into two independent channels:



Both channels are try/catch-wrapped; ② failing merely logs a warning. **The system is fully functional with the WS service switched off** — that is exactly what happens on Vercel, and why the deployment guide calls realtime "degraded to refresh-visible".

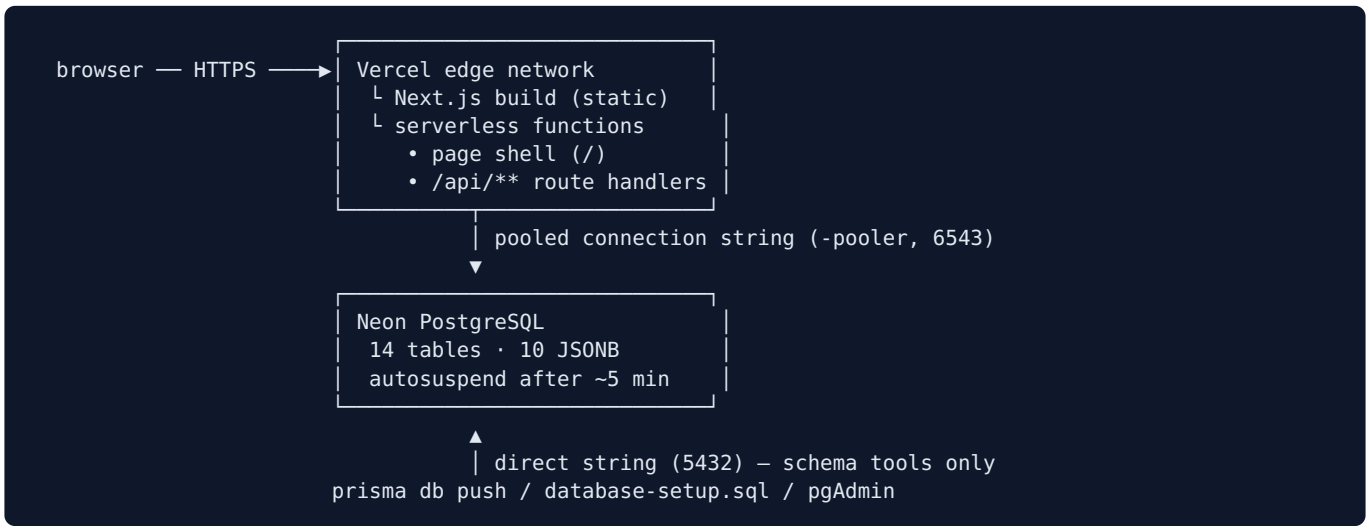
2. mini-services/notifications — the WS relay

A deliberately tiny, **database-free** Bun/Node service (`socket.io ^4.8.1`, one file).

Aspect	Detail
Listen	`PORT = 3003`, `socket.io` server with `path: '/'`, CORS `*`, ping 25 s / timeout 60 s
HTTP ingress	`POST /notify` — body `{ userId, …payload }` → `io.to('user:'+userId).emit('notification', payload)` → `{"ok":true}`; invalid JSON → 400; anything else 404
Client connect	browser hook `use-notification-socket.ts` connects to `/?XTransformPort=3003` (the dev gateway proxies by query param) with `transports: ['websocket','polling']`
Room join	client emits `auth { userId, token: 'dev' }` → server joins room `user:<id>`; **token is not validated** (placeholder — see notes)
Push path	Next.js server (the only broadcaster) → per-user rooms → `notification` event → client store `addNotification` (cap 50, unread badge recompute)
Resilience	client: socket.io auto-reconnect **plus** manual exponential backoff (1 s → 30 s cap); server: SIGTERM/SIGINT graceful close
Deploy	optional: Railway/Render/Fly (needs a long-running process). On Vercel it simply isn't deployed

Security note (honest): any client that knows a `userId` could join that room and receive its notification pushes early. The payload is the same data the user can fetch anyway, and the service is meant to run on a private network next to the app — but do not expose port 3003 publicly without adding JWT verification to the `auth` handler.

3. Production topology on Vercel + Neon



Why two connection strings (recap)

Serverless functions open many short-lived connections; Neon's **pooler** endpoint (`-pooler`, port 6543) is built for exactly that and is what `DATABASE_URL` points to. Operations that manage the schema (`db push`, pgAdmin) want the **direct** endpoint (`DIRECT_URL`, port 5432). Both are set as env vars at deploy time.

What Vercel runs

Step	Command / source	Notes
install	`npm install`	`postinstall: prisma generate` builds the typed client
build	`next build`	verified: 16 routes, all `/api/**` dynamic, pages static — no DB at build time
serve	Vercel functions	`PORT` managed by the platform

Environment variables (the only three)

`DATABASE_URL` (pooled) · `DIRECT_URL` (direct) · `JWT_SECRET` (long random). Changing them requires a **redeploy** to take effect.

Behavioural differences vs local dev

	Local (`npm run dev`)	Vercel
Realtime WS push	works (if mini-service runs on :3003)	falls back to polling/refresh
Cold start	none	first request after ~5 min idle wakes Neon (3–8 s)
`localhost:3003` notify fetch	succeeds	fails silently (by design)
Seeding	`GET /api/seed` in browser	identical (`/api/seed` , idempotent)
Free-tier DB pause	n/a	Neon: sleeps after ~5 min; Supabase: 7-day pause **prevented by the daily keep-alive**; Aiven: never sleeps

The daily keep-alive cron (`vercel.json` → `/api/keep-alive`)

`vercel.json` schedules exactly one Vercel Cron job: a daily `GET /api/keep-alive` (the Hobby plan's maximum frequency — which is enough). The endpoint runs `SELECT 1` through Prisma, i.e. real database activity. Its purpose is free-tier pause policy: Neon's ~5-minute sleep cannot be prevented by a daily ping, but **Supabase** only pauses after 7 completely idle days — so the daily ping keeps a free Supabase database awake forever — and **Aiven** never sleeps at all. The endpoint is public because Vercel Cron sends unauthenticated GETs; it leaks nothing (no data, `SELECT 1` only) and is safe to call from a browser too. Full switch-over steps for Supabase/Aiven live in DEPLOYMENT-GUIDE.md, Appendix E.

4. Local development topology (for completeness)

```
npm run dev (Next.js :3000)  ↔  any PostgreSQL (DATABASE_URL = DIRECT_URL locally)
        │
        └─ notify fetch → localhost:3003 (mini-services/notifications, `bun --hot index.ts`)
```

Locally both Prisma URLs point at the same database, so `db push` and the app share one string. A clean local DB is brought up either by Docker (`.env.example` has the exact command) or by pointing at Neon and pasting `prisma/database-setup.sql` in the cloud console — the deployment guide's Part 2 does the latter with zero local tools.

5. Operations quick reference

Task	How
Deploy an app change	push to GitHub → Vercel auto-deploys
Change a secret	Vercel → Settings → Environment Variables → **Redeploy**
Create tables on a fresh DB	paste `prisma/database-setup.sql` in Neon SQL Editor (or `npm run db:push`)
Load demo data	open `https://your-app/api/seed` once
Reset all data	`TRUNCATE … CASCADE` from the deployment guide FAQ Q7, then seed again
Inspect data	pgAdmin 4 (direct string + SSL Require) or Neon SQL Editor (`SELECT * FROM "TestCase"`)
Backups	Neon point-in-time (6 h free tier) / Neon branch / pgAdmin dump
Logs	Vercel dashboard → Deployments → Runtime Logs (`console.*` from routes appears here)

13 · File Index — every file, explained

> One entry per project file (line counts approximate). Grouped by directory; "→ doc" points at the deep-dive document. shadcn/ui primitives are grouped at the end since they are template stock.

Root & configuration

File	Lines	What it is
`package.json`	—	Scripts (`dev/build/start/lint`, `postinstall: prisma generate`, `db:*`), every dependency pin. → doc 01 §2
`package-lock.json`	—	Reproducible install for Vercel
`tsconfig.json`	—	TypeScript config (`@/*` → `src/*` path alias)
`next.config.ts`	—	Minimal; standalone output removed for Vercel
`vercel.json`	—	Vercel Cron schedule: daily `GET /api/keep-alive` (anti-pause ping for free-tier databases). → doc 12 §3
`tailwind.config.ts`	—	Legacy-style theme mapping (v4 CSS-first lives in globals.css)
`postcss.config.mjs`	—	Tailwind v4 postcss plugin
`eslint.config.mjs`	—	Flat ESLint config (next/core-web-vitals)
`components.json`	—	shadcn/ui generator settings
`CLAUDE.md`	—	Context file for AI coding assistants (architecture map, conventions, mandatory coding standards, DB naming rules, quirks, commands)
`REFACTORING-REPORT.md`	—	Full 2026-09 restructure report: 4-layer API + raw SQL migration, table/PK rename mapping, verification results
`.gitignore`	—	Keeps `.env*`, `/db/`, build output out of Git
`.env.example`	—	Documented template for the 3 env vars
`DEPLOYMENT-GUIDE.md`	—	Step-by-step online deployment (English)
`README.md`	—	Project overview + quick start
`CODE-QUALITY-ROADMAP.md`	—	Engineering improvement notes from development

prisma/

File	Lines	What it is
`schema.prisma`	324	The 14 models, relations, JSONB columns, indexes. → doc 02
`database-setup.sql`	410	Generated SQL that creates all tables — paste into Neon SQL Editor (deployment Part 2)

src/app — Next.js App Router

File	Lines	What it is
`layout.tsx`	41	Root server layout: metadata, Geist fonts, `<Providers>`, both toasters
`page.tsx`	7	The single page — renders `<AppShell/>` (the whole SPA)
`globals.css`	122	Tailwind v4 `@theme inline` tokens (oklch), dark variant, print-area helper

src/app/api — 13 route handlers (→ doc 04)

File	Lines	Endpoints
`route.ts`	4	`GET /api` health check
`auth/route.ts`	199	register / login / refresh / logout / me / profile / password
`seed/route.ts`	287	Idempotent demo-data seeder (GET+POST; GET added for browser seeding)
`keep-alive/route.ts`	43	Public `GET` → `SELECT 1`; hit daily by the Vercel Cron in `vercel.json` so free-tier DBs (Supabase) never pause. → doc 04 §14
`stats/route.ts`	67	`?action=dashboard` (6 parallel aggregates) / `?action=reports` (lead/admin)
`modules/route.ts`	332	tree/detail/create/update/delete + `?action=submit-review` (module review orchestration)
`test-cases/route.ts`	372	list/detail/history/reviews + create + versioned update + `?action=submit-review` + soft delete
`reviews/route.ts`	227	queue / all / detail + **verdict** (status machine + module aggregation)
`module-reviews/route.ts`	191	queue / all / detail + `?action=approve-all`
`bugs/route.ts`	243	list/detail/links + create + update (resolvedAt stamping) + link-tc + admin-only hard delete
`task-reports/route.ts`	227	my / summary / list / upsert-by-date / owner-only update
`activity-logs/route.ts`	75	filtered audit list + per-entity / per-user branches
`notifications/route.ts`	63	list / read-all / read-one / delete (all user-scoped)
`users/route.ts`	146	admin-only CRUD (invite, role, activate, soft delete)

src/components — app chrome

File	Lines	What it is
`providers.tsx`	49	`QueryInit` (React Query defaults) + `AuthGate` (session restore spinner). → doc 05 §2
`app-shell.tsx`	101	Sidebar + topbar + `CurrentView` switch over 14 views; routes to `AuthScreen` when logged out
`auth-screen.tsx`	189	Login/register split screen + demo account quick-fills. → doc 06
`layout/sidebar.tsx`	143	11 nav items with role visibility, reviews pending badge (30 s poll), collapse + mobile drawer
`layout/topbar.tsx`	176	Mobile menu button, notification bell popover (first 10, mark-read, deep links), user menu

src/components/views — the 14 "pages" (→ docs 06–10)

File	Lines	ViewKey	One-liner
`dashboard.tsx`	236	`dashboard`	KPI cards, status/severity charts, recent activity feed
`modules.tsx`	511	`modules`	Module tree browser, create/delete, submit module for review
`test-cases.tsx`	411	`test-cases`	Filterable table, create dialog (steps editor), CSV export
`test-case-detail.tsx`	626	`test-case-detail`	Read/edit modes, steps table, submit for review, soft delete, history/reviews/activity panels
`reviews.tsx`	253	`reviews`	Module-review / single-case / all-reviews queues with overdue detection
`review-detail.tsx`	347	`review-detail`	Read-only case + verdict dialog (flagged fields, mandatory comment)
`module-review-detail.tsx`	415	`module-review-detail`	Bulk workspace: stats, expandable cases, per-case verdicts, Approve All
`bugs.tsx`	492	`bugs`	dnd-kit Kanban + table, create dialog, detail sheet with inline edit
`tasks.tsx`	546	`tasks`	Daily report editor (task lines, entity links), manager user-switch, CSV export
`reports.tsx`	418	`reports`	Team weekly hours + status/priority/bug charts (lead/admin), print-to-PDF
`activity-log.tsx`	463	`activity-log`	Audit table, 6 filters, before/after diff dialog, cross-navigation
`notifications.tsx`	253	`notifications`	Inbox over the Zustand store: icons, read/unread, deep links, delete
`settings-users.tsx`	522	`settings-users`	Admin console: invite dialog, inline role switch, deactivate, filters
`settings-profile.tsx`	322	`settings-profile`	Account card, profile/notification prefs, change password

src/components/shared — tiny building blocks

File	Lines	What it is
`state.tsx`	96	`StateLoading / StateLoadingCards / StateSpinner / StateEmpty / StateError`
`pagination.tsx`	43	Shared Prev/Next pagination (used by test-cases)

src/hooks

File	Lines	What it is
`use-notification-socket.ts`	79	socket.io client → room auth → `notification` → store; backoff reconnect; seeds 50 items. → doc 12
`use-toast.ts`	193	shadcn toast queue (legacy — views use sonner)
`use-mobile.ts`	19	`matchMedia` breakpoint helper (template)

src/lib — the two backbones + helpers

File	Lines	What it is
`client/api.ts`	297	**Frontend data layer**: session storage, `request<T>()` with 401 auto-refresh, 12 API namespaces. → doc 05 §3
`auth.ts`	152	**Server auth**: JWT sign/verify, `getCurrentUser()` (cookie), bcrypt, `ROLE_HIERARCHY`, `PERMISSION_MATRIX` / `can()`, session create/revoke. → doc 03
`api.ts`	49	Response helpers: `ok/paginate/badRequest/unauthorized/forbidden/notFound/getQueryParams`
`db.ts`	12	Prisma singleton (HMR-safe via globalThis in dev)
`activity.ts`	79	`logActivity()` (snapshots + diff + IP/UA, never throws) + JSONB `parseJson/toJson`
`notifications.ts`	81	`createNotification()` (preference-gated insert) + fire-and-forget WS broadcast. → doc 12
`serializers.ts`	374	7 row normalisers guaranteeing JSONB parsed exactly once (+ `TaskItem` type)
`format.ts`	145	All color maps + date/duration/text formatters (exact mappings in doc 05 §6)
`errors.ts`	63	Client `ApiError` → friendly toast message mapping; `handleMutation`
`utils.ts`	6	`cn()` Tailwind class merger

src/store & src/types

File	Lines	What it is
`store/index.ts`	142	Zustand: `useAuthStore` (session), `useViewStore` (router replacement), `useNotificationsStore`. → doc 05 §2
`types/index.ts`	245	All domain types & string-enum unions shared by views (`TestCase`, `ReviewRequest`, `Paginated<T>`, …)

mini-services/notifications

File	Lines	What it is
`index.ts`	95	socket.io server on :3003; `POST /notify` → per-user rooms; `auth` / `ping` protocol; graceful shutdown. → doc 12
`package.json`	10	`tcms-notifications-ws`, single dep `socket.io`, `bun --hot` dev script

src/components/ui — 40 shadcn/ui primitives (template stock)

`accordion, alert-dialog, alert, aspect-ratio, avatar, badge, breadcrumb, button, calendar, card, carousel, chart, checkbox, collapsible, command, context-menu, dialog, drawer, dropdown-menu, form, hover-card, input-otp, input, label, menubar, navigation-menu, pagination, popover, progress, radio-group, resizable, scroll-area, select, separator, sheet, sidebar, skeleton, slider, sonner, switch, table, tabs, textarea, toast, toaster, toggle-group, toggle, tooltip`

All are standard shadcn/ui Radix wrappers customized only by the theme tokens in `globals.css`. The views lean on ~20 of them (button, card, dialog, select, table, badge, tabs, sheet, popover, dropdown-menu, switch, checkbox, calendar, alert, alert-dialog, avatar, tooltip, scroll-area, textarea, input, separator); the rest are unused conveniences that ship with the template. `ui/sidebar.tsx` (726 lines) and `ui/chart.tsx` (353) are the largest but likewise stock.